

GenMatch: An End-to-End Generative Matching Framework for Micro-View Order-Dispatching in Ride-Hailing

Chuang Liu*
Didi Chuxing
Hangzhou, China
chuangliu@didiglobal.com

Yuxueqing Zhang
Didi Chuxing
Beijing, China
zhangyuxueqing@didiglobal.com

Tengfei Lyu
The Hong Kong University of Science
and Technology (Guangzhou)
Guangzhou, China
tlyu077@connect.hkust-gz.edu.cn

Zirui Yuan
The Hong Kong University of Science
and Technology (Guangzhou)
Guangzhou, China
zyuan779@connect.hkust-gz.edu.cn

WeiQi Hu
Didi Chuxing
Beijing, China
huweiqi@didiglobal.com

Yanghan Cheng
Didi Chuxing
Beijing, China
chengyanghan@didiglobal.com

Ming Wang
Didi Chuxing
Beijing, China
nicholaswangming@didiglobal.com

Li Ma
Didi Chuxing
Beijing, China
malimarey@didiglobal.com

Zihao Lu†
Didi Chuxing
Beijing, China
luzihao@didiglobal.com

Abstract

Micro-View Order-Dispatching assigns available drivers to passenger orders within each dispatch batch. It is critical to the service quality and operational efficiency of ride-hailing platforms. Mainstream industrial solutions follow a multi-stage paradigm consisting of model prediction, value calculation, and dispatch matching. Although dispatch quality is determined by the final batch-level assignment, these stages optimize different intermediate objectives. This creates cross-stage objective inconsistency, so improving any single stage does not necessarily improve the overall dispatch result. Generative modeling offers a natural solution by mapping the system context directly to the final output. Motivated by this capability, we formulate Micro-View Order-Dispatching as a generative matching problem and propose an end-to-end Generative Matching framework (GenMatch), the first generative framework for this task to be deployed in a real-world production environment. However, applying generative modeling to this problem introduces three challenges. First, the model must encode an entire dispatch batch, but each batch forms a dynamic sparse bipartite graph, requiring efficient structured batch-level encoding. Second, replacing the hand-crafted value function requires learning unified business utility from heterogeneous feedback. Third, directly generating an assignment requires tracking the evolving matching state because every selected order-driver pair changes the remaining feasible candidates. GenMatch addresses these challenges through a Context-Aware Bipartite Encoder, a Business-Aware Utility Learner, and a State-Aware Pointer Decoder. Extensive offline evaluations and online A/B tests in five cities across DiDi’s international ride-hailing markets demonstrate consistent improvements over competitive baselines, confirming the effectiveness and practicality of GenMatch for industrial order-dispatching.

*First author.

†Corresponding author.

Keywords

Ride-Hailing; Micro-View Order-Dispatching; Generative Matching; Sequential Decision Making;

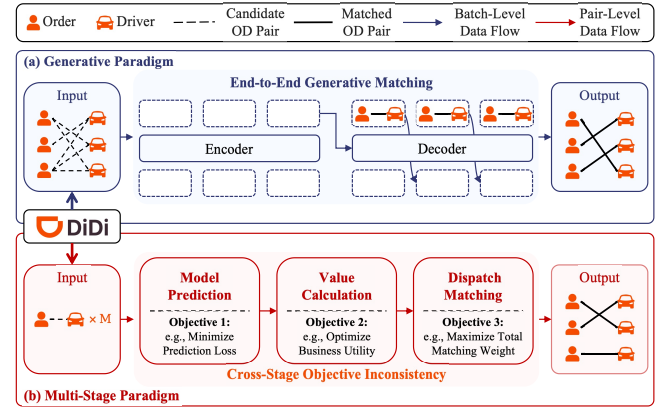


Figure 1: Comparison of (a) the end-to-end generative paradigm and (b) the conventional multi-stage paradigm for Micro-View Order-Dispatching.

1 Introduction

Order-dispatching refers to assigning passengers’ orders to available drivers in real time. It directly affects passenger and driver experience, which in turn influences platform revenue. Therefore, it is a core process of ride-hailing platforms. Existing studies take two complementary views [34]. The Macro-View values and coordinates current decisions under long-term supply–demand evolution [15, 22, 29]. In contrast, the Micro-View considers short-term, real-time assignment within a city. At each fixed dispatch interval,

the system collects available orders and drivers; every feasible order-driver combination forms an order-driver (OD) pair, and together they constitute a dispatch batch. Selecting one-to-one assignments from this batch under strict latency constraints is termed Micro-View Order-Dispatching (MICOD).

Mainstream industrial MICOD solutions follow the multi-stage paradigm in Figure 1(b), operating on a single feasible OD pair (pair-level) and the entire dispatch batch (batch-level). They predict pair-level business signals, such as the probabilities of driver answer (DA), passenger cancellation after answer (PCAA), and driver cancellation after answer (DCAA); aggregate them into matching weights with a hand-crafted value function; and apply a batch-level solver such as Kuhn–Munkres matching [10, 14]. Although practical, separately optimizing these stages creates *cross-stage objective inconsistency*: better intermediate predictions or weights need not improve the final assignment. Generative models address the same issue by mapping context directly to final outputs in search and query suggestion [1, 5], recommendation [3, 40], and advertising [30]. We therefore formulate MICOD as end-to-end generation of an assignment from the complete dispatch batch.

This formulation must recover the three capabilities of the replaced pipeline: representing candidates, combining heterogeneous business objectives, and constructing a feasible batch-level matching. This yields three domain-specific challenges.

C1: Structured batch-level encoding of a dynamic sparse bipartite graph. Unlike independent pair-level models, a generative model must encode the entire dispatch batch. Its bipartite graph varies in orders, drivers, and feasible OD pairs and is highly sparse, requiring structured batch-level encoding that preserves online computational efficiency.

C2: Learning unified business utility from heterogeneous feedback. Replacing manually tuned value rules requires learning business utility directly. Yet generation targets do not fully express outcomes such as DA, PCAA, and DCAA, whose different semantics and directions must be integrated into stable, unified guidance.

C3: Generating batch-level assignments under an evolving matching state. Dispatch is evaluated by its joint assignment rather than individual pair scores. During generation, selecting one OD pair invalidates every pair sharing its order or driver and changes the remaining opportunities, requiring an evolving state that supports feasible, batch-coordinated decisions.

To address them, we propose Generative Matching (GenMatch), which maps a dispatch batch directly to an assignment sequence through the encoder–decoder paradigm in Figure 1(a). Its Context-Aware Bipartite Encoder performs sparse message passing over feasible OD edges; its Business-Aware Utility Learner uses auxiliary supervision to derive business guidance from heterogeneous outcomes; and its State-Aware Pointer Decoder tracks selected and residual candidates while dynamically masking infeasible pairs. Together, they support end-to-end generation under MICOD’s structural, business, and matching constraints.

The main contributions are summarized as follows.

- To the best of our knowledge, we propose the first end-to-end generative framework for MICOD deployed in a real-world

production environment. It directly generates the assignment from an entire dispatch batch and avoids the cross-stage objective inconsistency.

- We develop three components for generative matching: a Context-Aware Bipartite Encoder to efficiently encode dynamic sparse bipartite graphs, a Business-Aware Utility Learner to learn unified utility from heterogeneous feedback, and a State-Aware Pointer Decoder to generate feasible assignments under an evolving matching state.
- We conduct extensive offline evaluations and online A/B tests in five cities across DiDi’s international ride-hailing markets. The results demonstrate consistent improvements over competitive baselines and confirm the effectiveness and practical value of GenMatch in production systems.

2 Related Work

2.1 Ride-Hailing Order-Dispatching

Effective order-dispatching improves service quality and platform efficiency. Macro-View studies optimize long-term demand–supply dynamics through regional value learning [21, 22, 29], online or offline reinforcement learning [18, 37], knowledge-enhanced dispatch control [7], and multi-agent regional cooperation [9, 25, 31].

Micro-View studies make decisions within the current dispatch batch through three paradigms. Policy-based methods such as CoRide and CoopRide produce regional or grid-level actions rather than complete OD-pair assignments [9, 25]. D2SN sequentially selects OD pairs or hold actions using a two-layer Markov decision process and an encoder–decoder reinforcement-learning policy [34], but lacks explicit OD-pair interaction modeling and direct supervision from heterogeneous business outcomes. The industrial multi-stage paradigm predicts pair-level signals, calculates matching weights, and applies Greedy [38], Kuhn–Munkres [10, 14], or Gale–Shapley [4] matching. Related systems use deep graph learning for constrained matchmaking and courier pooling [13, 20], but address generic matchmaking or many-to-one assignment rather than one-to-one MICOD. GenMatch instead unifies batch encoding, business-utility learning, and feasible assignment generation.

2.2 Generative Modeling

Generative modeling produces a target sequence autoregressively from its context instead of scoring only predefined outputs, supporting large or variable output spaces. Industrial systems have applied this formulation to item generation and large-scale sequential recommendation [16, 36], food-delivery and e-commerce recommendation [6, 42], and multi-business, local-life, and advertising scenarios [11, 28, 30]. Distributed training systems further scale generative recommendation to industrial workloads [26]. Recent extensions combine behavioral and semantic information, generate long semantic identifiers in parallel, or apply semantic identifiers to next-point-of-interest recommendation [8, 24, 27]. Other studies address the training–inference gap through prefix-aware optimization or combine diffusion with knowledge-graph reasoning [33, 39]. Reasoning-augmented language models further support generative next-point-of-interest recommendation [41]. Non-autoregressive

generation has also been explored for reranking, while large language models connect quality-aware ranking with candidate generation at web scale [17, 19].

Generative methods also reduce cross-stage objective inconsistency by replacing retrieval and ranking with direct recommendation generation [3, 40], unifying e-commerce search and query suggestion [1, 2, 5], or jointly training retrieval and ranking [12]. Multi-stage alignment further learns preferences from clicks for generative query suggestion [32], while reinforcement learning improves relevance within generative search ranking [35]. Pointer Networks show that structured solutions can be generated by selecting elements from a variable-size input set [23]. MICOD shares this structure, but its candidates form a dynamic sparse bipartite graph whose feasible set changes after every selection. GenMatch extends pointer generation to construct a complete one-to-one matching from such a dispatch batch.

3 Preliminary

In this section, we first describe the MICOD process and introduce the corresponding business concepts and notation. We then formulate MICOD as a batch-level matching problem and present its generative formulation.

MICOD Business Process. A ride-hailing platform triggers dispatch at fixed intervals. At dispatch step t , the platform first collects the currently available passenger orders and drivers. Let $\mathcal{O}_t = \{o_1, \dots, o_{N_t^o}\}$ and $\mathcal{D}_t = \{d_1, \dots, d_{N_t^d}\}$ denote the order and driver sets, respectively. Their sizes vary across dispatch steps. The upstream system then filters infeasible combinations according to pickup distance, service range, and other business rules. The remaining feasible OD pairs form the candidate set $\mathcal{E}_t \subseteq \mathcal{O}_t \times \mathcal{D}_t$. Together, the orders, drivers, and candidate OD pairs constitute a *dispatch batch*, represented as a dynamic sparse bipartite graph

$$\mathcal{G}_t = (\mathcal{O}_t, \mathcal{D}_t, \mathcal{E}_t), \quad (1)$$

where each candidate edge $e_{ij} = (o_i, d_j) \in \mathcal{E}_t$ denotes a feasible OD pair. Let $P_t = |\mathcal{E}_t|$ be the number of candidate OD pairs. We index these edges as $\mathcal{E}_t = \{e_1, \dots, e_{P_t}\}$, where each e_r corresponds to an edge e_{ij} . Each order, driver, and candidate edge is associated with a feature vector $\mathbf{x}_i^o \in \mathbb{R}^{d_o}$, $\mathbf{x}_j^d \in \mathbb{R}^{d_d}$, and $\mathbf{x}_{ij}^e \in \mathbb{R}^{d_e}$, respectively. These feature vectors encode spatiotemporal, behavioral, and business signals (e.g., pickup time and platform revenue).

The dispatch system takes the complete dispatch batch $\mathcal{G}_t$ as input and selects a set of OD pairs $\mathcal{M}_t \subseteq \mathcal{E}_t$ for assignment. Each selected pair is broadcast to its driver and enters a stage-wise service process. The driver decides whether to answer. After an answer, the passenger or driver may cancel; the trip is completed only if neither side cancels. This process produces DA, PCAA, DCAA, and trip-completion outcomes. For each broadcast pair e_{ij} , $\mathbf{y}_{ij}$ records these events. The multi-task labels follow this service dependency, and downstream events can occur only after driver answer. Unbroadcast candidates have no observed service-process labels and are excluded from the auxiliary loss. Let $\mathcal{C}_t \subseteq \mathcal{M}_t$ denote the set of OD pairs that complete their trips. The completed set $\mathcal{C}_t$ is later used to construct the target sequence for generative training. Candidate retrieval and feasibility filtering are provided by the existing upstream system and are outside the scope of this work.

MICOD Problem Formulation. Let $U(\mathcal{M}_t; \mathcal{G}_t)$ denote the business utility of assignment $\mathcal{M}_t$ within dispatch batch $\mathcal{G}_t$. It captures the joint quality of the assignment through outcomes such as DA, PCAA, DCAA, trip completion, pickup time, and platform revenue. Because selected OD pairs compete for shared orders and drivers, this utility is defined over the complete assignment rather than independent pairs. The MICOD objective is to find the feasible matching with the highest batch-level assignment utility:

$$\mathcal{M}_t^* = \arg \max_{\mathcal{M}_t \in \mathcal{F}(\mathcal{G}_t)} U(\mathcal{M}_t; \mathcal{G}_t), \quad (2)$$

where $\mathcal{F}(\mathcal{G}_t)$ is the set of feasible matchings over $\mathcal{G}_t$. Each order and each driver can appear in at most one selected OD pair:

$$\begin{aligned} \sum_j \mathbb{I}(o_i, d_j) \in \mathcal{M}_t &\leq 1, \quad \forall o_i \in \mathcal{O}_t, \\ \sum_i \mathbb{I}(o_i, d_j) \in \mathcal{M}_t &\leq 1, \quad \forall d_j \in \mathcal{D}_t. \end{aligned} \quad (3)$$

Here, $\mathbb{I}(\cdot)$ is the indicator function. The matching must also be produced within a strict online latency budget. The conventional multi-stage paradigm approximates this objective by predicting pair-level signals, converting them into matching weights through a hand-crafted value function, and applying a matching solver. In this work, end-to-end means that model prediction, value calculation, and dispatch matching are replaced by one jointly trained model.

Generative MICOD Formulation. GenMatch constructs the assignment directly through sequential generation over the candidate OD pairs. An assignment $\mathcal{M}_t$ can be represented by an ordered generation sequence

$$\mathcal{Y}_t = \left(e_t^{(1)}, e_t^{(2)}, \dots, e_t^{(K_t)} \right), \quad (4)$$

where $K_t = |\mathcal{M}_t|$ and the set of generated OD pairs equals $\mathcal{M}_t$. The sequence specifies how the matching is constructed; the final dispatch result is the resulting set rather than a generation order. At generation step k , the model selects one candidate $e_t^{(k)}$ from the residual graph $\mathcal{G}_t^{(k)}$. All candidates sharing its order or driver then become infeasible and are removed before the next step. The probability of the complete assignment sequence is factorized as

$$\Pr(\mathcal{Y}_t | \mathcal{G}_t) = \prod_{k=1}^{K_t} \Pr\left(e_t^{(k)} | \mathcal{G}_t^{(k)}, e_t^{(<k)}\right), \quad (5)$$

where $e_t^{(<k)}$ denotes the selected OD pairs. GenMatch learns to map the complete dispatch batch directly to a feasible assignment while conditioning each decision on the evolving matching state. The next section presents how this formulation is instantiated.

4 Methods

Figure 2 presents the Context-Aware Bipartite Encoder, Business-Aware Utility Learner, and State-Aware Pointer Decoder in panels (a), (b), and (c), respectively. They model matching and competition information, learn business-aware candidate representations and utility logits, and generate assignments under the evolving matching state. We follow Section 3 and omit t when clear; e_r denotes the r -th candidate edge, whereas e_{ij} emphasizes its endpoints.

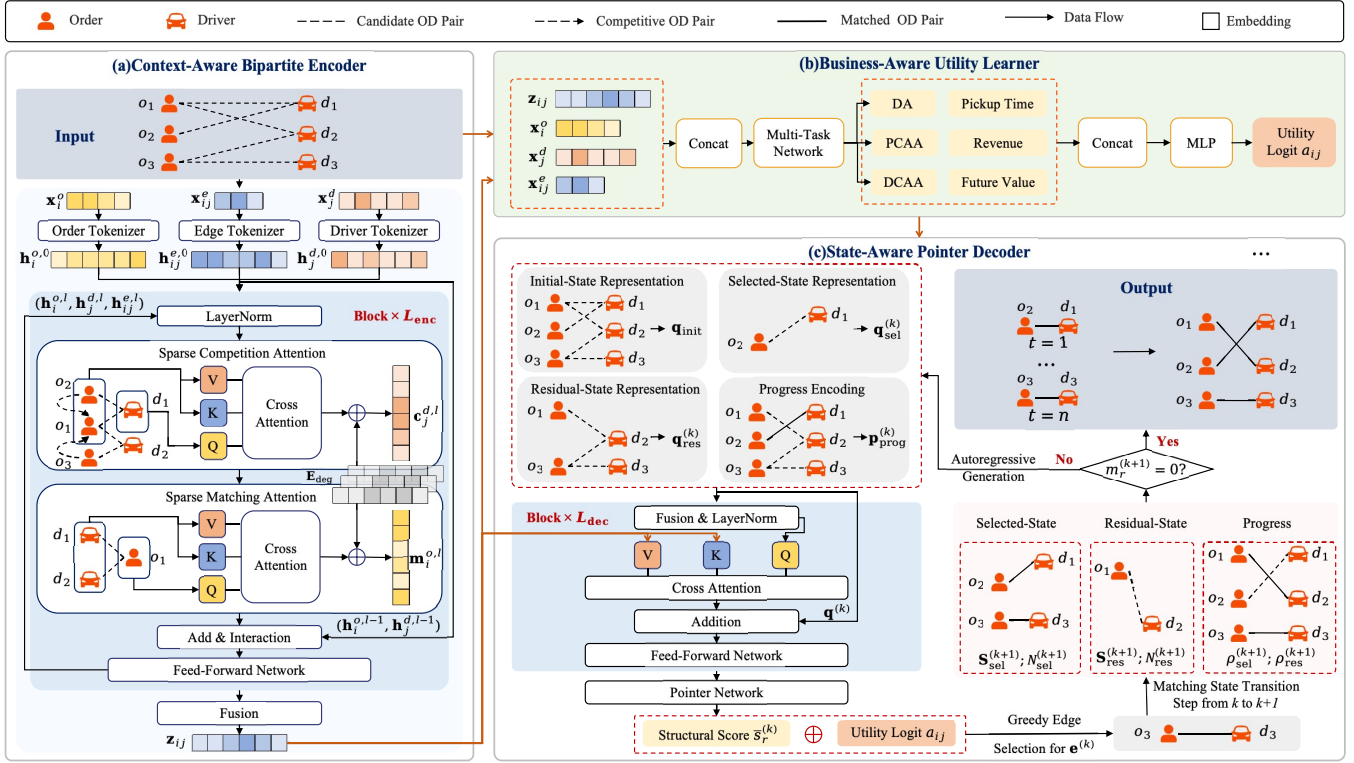


Figure 2: GenMatch architecture: (a) Context-Aware Bipartite Encoder, (b) Business-Aware Utility Learner, and (c) State-Aware Pointer Decoder.

4.1 Context-Aware Bipartite Encoder

The suitability of an OD pair depends not only on its own features but also on the other candidates sharing its order or driver. The Context-Aware Bipartite Encoder in Figure 2(a) therefore models batch-level matching and competition relationships over the complete candidate graph. It first summarizes candidates competing at each endpoint, then evaluates potential matches under this competitive context.

4.1.1 Input Representation. The encoder takes $\mathcal{G}_t$ and transforms its raw order, driver, and candidate-edge features $\mathbf{x}_i^o$, $\mathbf{x}_j^d$, and $\mathbf{x}_{ij}^e$ with separate tokenizers. The resulting d -dimensional embeddings $\mathbf{h}_i^{o,0}$, $\mathbf{h}_j^{d,0}$, and $\mathbf{h}_{ij}^{e,0}$ initialize the sparse bipartite encoder.

4.1.2 Sparse Matching and Competition Modeling. The encoder stacks L_{enc} sparse layers. Layer $l \in \{1, \dots, L_{\text{enc}}\}$ maps order, driver, and edge states with superscript $l-1$ to those with superscript l , first summarizing competitors and then evaluating matches under that context. Let $\mathcal{N}(o_i)$ and $\mathcal{N}(d_j)$ denote the candidate neighbors of order o_i and driver d_j .

For each edge e_{ij} incident to driver d_j , $\mathbf{r}_{i \rightarrow j}^l = \mathbf{h}_i^{o,l-1} + \mathbf{h}_{ij}^{e,l-1}$ carries order and edge information. Let $\mathbf{W}_{q,c}^{d,l}$, $\mathbf{W}_{k,c}^{d,l}$, and $\mathbf{W}_{v,c}^{d,l}$ be the competition-attention projections. We also use a shared degree embedding matrix $\mathbf{E}_{\text{deg}} \in \mathbb{R}^{(D_{\text{max}}+1) \times d}$, where D_{max} is the maximum retained candidate degree; its indexed row represents neighborhood size. We show one head below, while the implementation

concatenates multiple heads. Aggregating $\mathbf{r}_{i \rightarrow j}^l$ over $\mathcal{N}(d_j)$ gives competition information $\mathbf{c}_j^{d,l}$:

$$\begin{aligned} \beta_{ij}^{d,l} &= \text{softmax}_{o_i \in \mathcal{N}(d_j)} \left(\frac{(\mathbf{W}_{q,c}^{d,l} \mathbf{h}_j^{d,l-1})^\top (\mathbf{W}_{k,c}^{d,l} \mathbf{r}_{i \rightarrow j}^l)}{\sqrt{d}} \right), \\ \mathbf{c}_j^{d,l} &= \sum_{o_i \in \mathcal{N}(d_j)} \beta_{ij}^{d,l} \mathbf{W}_{v,c}^{d,l} \mathbf{r}_{i \rightarrow j}^l + \mathbf{E}_{\text{deg}}[|\mathcal{N}(d_j)|]. \end{aligned} \quad (6)$$

Attention captures neighborhood composition, while the degree embedding preserves its size. Order-side competition information $\mathbf{c}_i^{o,l}$ is computed symmetrically from $\mathbf{r}_{j \rightarrow i}^l = \mathbf{h}_j^{d,l-1} + \mathbf{h}_{ij}^{e,l-1}$ over $\mathcal{N}(o_i)$.

We incorporate competition into each candidate representation as $\mathbf{z}_{j \rightarrow i}^l = \mathbf{h}_j^{d,l-1} + \mathbf{h}_{ij}^{e,l-1} + \mathbf{c}_j^{d,l}$, with $\mathbf{z}_{i \rightarrow j}^l$ defined symmetrically. Using matching-attention projections $\mathbf{W}_{q,m}^{o,l}$, $\mathbf{W}_{k,m}^{o,l}$, and $\mathbf{W}_{v,m}^{o,l}$, their aggregation over drivers produces matching information $\mathbf{m}_i^{o,l}$:

$$\begin{aligned} \alpha_{ij}^{o,l} &= \text{softmax}_{d_j \in \mathcal{N}(o_i)} \left(\frac{(\mathbf{W}_{q,m}^{o,l} \mathbf{h}_i^{o,l-1})^\top (\mathbf{W}_{k,m}^{o,l} \mathbf{z}_{j \rightarrow i}^l)}{\sqrt{d}} \right), \\ \mathbf{m}_i^{o,l} &= \sum_{d_j \in \mathcal{N}(o_i)} \alpha_{ij}^{o,l} \mathbf{W}_{v,m}^{o,l} \mathbf{z}_{j \rightarrow i}^l + \mathbf{E}_{\text{deg}}[|\mathcal{N}(o_i)|]. \end{aligned} \quad (7)$$

Driver-side matching information $\mathbf{m}_j^{d,l}$ is computed symmetrically. Competition attention relates candidates sharing an endpoint; matching attention relates a node to its counterparts. Both use $\mathbf{E}_{\text{deg}}$ to preserve neighborhood size and operate only on candidate edges.

Let LN denote layer normalization, and let FFN_o^l , FFN_d^l , and FFN_e^l denote the order-, driver-, and edge-side feed-forward networks in layer l , respectively. Each block includes its residual connection and layer normalization. The order state is therefore updated as

$$\mathbf{h}_i^{o,l} = \text{FFN}_o^l(\mathbf{h}_i^{o,l-1} + \mathbf{m}_i^{o,l}). \quad (8)$$

The driver state $\mathbf{h}_j^{d,l}$ is updated symmetrically from $\mathbf{h}_j^{d,l-1}$ and $\mathbf{m}_j^{d,l}$.

The encoder also updates each edge state to retain OD-pair-specific information while incorporating matching information from both endpoints. Let $\odot$ denote element-wise multiplication and $\tilde{\mathbf{h}}_{ij}^{e,l}$ denote the interaction representation. For candidate edge e_{ij} , we combine its preceding state with the sum, product, and absolute difference of the order-side and driver-side matching information:

$$\tilde{\mathbf{h}}_{ij}^{e,l} = \mathbf{h}_{ij}^{e,l-1} + \mathbf{m}_i^{o,l} + \mathbf{m}_j^{d,l} + \mathbf{m}_i^{o,l} \odot \mathbf{m}_j^{d,l} + |\mathbf{m}_i^{o,l} - \mathbf{m}_j^{d,l}|. \quad (9)$$

The edge-side block produces $\mathbf{h}_{ij}^{e,l} = \text{FFN}_e^l(\tilde{\mathbf{h}}_{ij}^{e,l})$. After L_{enc} layers, the final pair representation is

$$\mathbf{z}_{ij} = \text{LN}(\mathbf{h}_i^{o,L} + \mathbf{h}_j^{d,L} + \mathbf{h}_{ij}^{e,L}). \quad (10)$$

The pair representation in Eq. (10) serves as candidate memory for the Business-Aware Utility Learner and State-Aware Pointer Decoder.

4.2 Business-Aware Utility Learner

Candidate selection requires reliable representations and explicit business-value guidance. The Business-Aware Utility Learner in Figure 2(b) therefore supervises candidate memory $\mathbf{z}_{ij}$ with stage-wise service outcomes and estimates utility from these outcomes and value signals. These two functions are complementary: auxiliary supervision improves candidate memory and injects service behavior, while the utility logit directly guides subsequent selection.

4.2.1 Stage-Wise Behavioral Outcome Learning. Because completion alone cannot identify where an unsuccessful service failed, we use the stage-wise outcomes defined in Section 3 for fine-grained supervision.

For OD pair (o_i, d_j) , let $\mathbf{v}_{ij} = [\mathbf{z}_{ij}; \mathbf{x}_i^o; \mathbf{x}_j^d; \mathbf{x}_{ij}^e]$, where $[\cdot]$ denotes concatenation. A multi-task learning (MTL) network predicts DA, PCAA, and DCAA probabilities $\hat{p}_{ij}^{\text{DA}}$, $\hat{p}_{ij}^{\text{PCAA}}$, and $\hat{p}_{ij}^{\text{DCAA}}$ under their service-stage dependencies. This supervision injects service behavior into $\mathbf{z}_{ij}$.

4.2.2 Business Utility Estimation. The stage-wise predictions describe whether an OD pair is likely to become an effective service, but not how much business value it may produce. Actual dispatch decisions must also consider pickup time, platform revenue, future value, and other business signals. Let $\mathbf{b}_{ij}$ denote these business-value fields selected from the raw edge features $\mathbf{x}_{ij}^e$. We concatenate them with the three stage-wise predictions as $\mathbf{b}_{ij}^{\text{all}} = [\hat{p}_{ij}^{\text{DA}}; \hat{p}_{ij}^{\text{PCAA}}; \hat{p}_{ij}^{\text{DCAA}}; \mathbf{b}_{ij}]$ and use a multi-layer perceptron (MLP) to

estimate the utility logit:

$$a_{ij} = \text{MLP}_{\text{biz}}(\mathbf{b}_{ij}^{\text{all}}). \quad (11)$$

The resulting a_{ij} is a candidate-specific and step-independent utility logit. It provides explicit business-value guidance to the subsequent matching process.

4.3 State-Aware Pointer Decoder

Each selection changes both feasibility and the remaining matching opportunities. The State-Aware Pointer Decoder in Figure 2(c) therefore conditions on the initial candidates, selected matching, residual candidates, and matching progress. The initial state anchors the original decision space, while the selected and residual states describe committed decisions and remaining opportunities.

4.3.1 Matching-State Representation. Because the final assignment is a set, we represent its evolving state with three order-invariant sets: initial, selected, and residual candidates. Let $\{\mathbf{z}_r\}_{r=1}^{P_t}$ be the pair representations and $m_r^{(k)} \in \{0, 1\}$ the selectability mask at step k , where $m_r^{(k)} = 1$ means that e_r remains selectable.

Each matching state is maintained by two state statistics: its representation sum and cardinality. The initial-state statistics are $\mathbf{S}_{\text{init}} = \sum_{r=1}^{P_t} \mathbf{z}_r$ and $N_{\text{init}} = P_t$. If $r^{(s)}$ denotes the candidate selected at step s , the selected-state statistics are $\mathbf{S}_{\text{sel}}^{(k)} = \sum_{s \leq k} \mathbf{z}_{r^{(s)}}$ and $N_{\text{sel}}^{(k)} = k - 1$. The residual-state statistics are $\mathbf{S}_{\text{res}}^{(k)} = \sum_{r=1}^{P_t} m_r^{(k)} \mathbf{z}_r$ and $N_{\text{res}}^{(k)} = \sum_{r=1}^{P_t} m_r^{(k)}$.

Dividing each nonempty sum by its cardinality gives $\mathbf{q}_{\text{init}} = \mathbf{S}_{\text{init}}/N_{\text{init}}$, $\mathbf{q}_{\text{sel}}^{(k)} = \mathbf{S}_{\text{sel}}^{(k)}/N_{\text{sel}}^{(k)}$, and $\mathbf{q}_{\text{res}}^{(k)} = \mathbf{S}_{\text{res}}^{(k)}/N_{\text{res}}^{(k)}$, representing the original batch, selected matching, and remaining opportunities. Before any selection, the selected set is empty, so we set $\mathbf{q}_{\text{sel}}^{(k)}$ to a learnable vector $\mathbf{e}_{\text{empty}}$.

Set averaging loses cardinality. We restore it with selected and residual ratios $\rho_{\text{sel}}^{(k)} = N_{\text{sel}}^{(k)}/\min(N_o^o, N_t^d)$ and $\rho_{\text{res}}^{(k)} = N_{\text{res}}^{(k)}/P_t$, normalized by the maximum matching size and initial candidate count. Because batch size is large and dynamic, we scale two learnable d -dimensional vectors $\mathbf{e}_{\text{sel}}$ and $\mathbf{e}_{\text{res}}$ rather than use a progress-indexed matrix. The resulting progress encoding and query are

$$\begin{aligned} \mathbf{p}_{\text{prog}}^{(k)} &= \rho_{\text{sel}}^{(k)} \mathbf{e}_{\text{sel}} + \rho_{\text{res}}^{(k)} \mathbf{e}_{\text{res}}, \\ \mathbf{q}^{(k)} &= \text{LN}(\mathbf{W}_{\text{init}} \mathbf{q}_{\text{init}} + \mathbf{W}_{\text{sel}} \mathbf{q}_{\text{sel}}^{(k)} + \mathbf{W}_{\text{res}} \mathbf{q}_{\text{res}}^{(k)} + \mathbf{p}_{\text{prog}}^{(k)}). \end{aligned} \quad (12)$$

Here, $\mathbf{W}_{\text{init}}$, $\mathbf{W}_{\text{sel}}$, and $\mathbf{W}_{\text{res}}$ are learnable projections; $\mathbf{p}_{\text{prog}}^{(k)}$ restores the scale information lost by averaging.

4.3.2 Generative Pointer Distribution. The query $\mathbf{q}^{(k)}$ is processed by L_{dec} cross-attention layers over the pair representations, producing the decoder state $\mathbf{h}^{(k)}$. Let P_{ptr} denote the number of pointer heads, and let $\mathbf{W}_q^{(p)}$ and $\mathbf{W}_k^{(p)}$ be the learnable query and key projections of head p . Each head produces a structural matching score $s_r^{(k,p)}$; the final structural score $\bar{s}_r^{(k)}$ is their mean:

$$\begin{aligned} s_r^{(k,p)} &= \frac{(\mathbf{W}_q^{(p)} \mathbf{h}^{(k)})^\top (\mathbf{W}_k^{(p)} \mathbf{z}_r)}{\sqrt{d}}, \\ \bar{s}_r^{(k)} &= \frac{1}{P_{\text{ptr}}} \sum_{p=1}^{P_{\text{ptr}}} s_r^{(k,p)}. \end{aligned} \quad (13)$$

Let $\mathcal{R}^{(k)} = \{r \mid m_r^{(k)} = 1\}$ denote the indices of feasible candidates. For candidate e_r , let a_r denote its utility logit; specifically, $a_r = a_{ij}$ when $e_r = e_{ij}$. We denote its conditional generation probability at step k by $p_r^{(k)}$. The decoder combines the step-dependent structural score with this step-independent utility logit:

$$p_r^{(k)} = \frac{\exp(\bar{s}_r^{(k)} + a_r)}{\sum_{r' \in \mathcal{R}^{(k)}} \exp(\bar{s}_{r'}^{(k)} + a_{r'})}, \quad r \in \mathcal{R}^{(k)}. \quad (14)$$

Here, a_r captures business utility, while $\bar{s}_r^{(k)}$ measures compatibility with the current matching state.

At inference step k , the decoder performs greedy edge selection by choosing the candidate with the highest generation probability:

$$r^{(k)} = \arg \max_{r \in \mathcal{R}^{(k)}} p_r^{(k)}, \quad e^{(k)} = e_{r^{(k)}}. \quad (15)$$

The selected edge $e^{(k)}$ is appended to the generated assignment, after which the matching state is updated for the next step.

4.3.3 Matching-State Transition. After selecting $e^{(k)} = e_{r^{(k)}}$, let the blocked set $\mathcal{B}^{(k)}$ contain it and all selectable pairs sharing its order or driver. The state statistics update incrementally as

$$\begin{aligned} S_{\text{sel}}^{(k+1)} &= S_{\text{sel}}^{(k)} + \mathbf{z}_{r^{(k)}}, \\ N_{\text{sel}}^{(k+1)} &= N_{\text{sel}}^{(k)} + 1, \\ S_{\text{res}}^{(k+1)} &= S_{\text{res}}^{(k)} - \sum_{r \in \mathcal{B}^{(k)}} \mathbf{z}_r, \\ N_{\text{res}}^{(k+1)} &= N_{\text{res}}^{(k)} - |\mathcal{B}^{(k)}|. \end{aligned} \quad (16)$$

We also set $m_r^{(k+1)} = 0$ for $r \in \mathcal{B}^{(k)}$ and retain all other mask values, enforcing one-to-one matching.

4.4 Model Training

4.4.1 Target Sequence Construction. Rather than distill production decisions, we construct supervision from the completed OD-pair set C_t , whose pairs were answered, not canceled, and completed.

Let $K_t^* = |C_t|$. Because assignment order has no business meaning, each training step uniformly permutes C_t . We denote the resulting target sequence by $\mathcal{Y}_t^* = (e_{t,*}^{(1)}, \dots, e_{t,*}^{(K_t^*)})$. This randomization prevents the decoder from fitting an arbitrary order.

Only completed pairs form targets, but all selectable candidates remain in the denominator of Eq. (14) as contrastive alternatives. Thus, GenMatch learns completion patterns from observed targets. At inference, generation continues while $\sum_{r=1}^{P_t} m_r^{(k)} > 0$, allowing GenMatch to select unbroadcast candidates exhibiting similar patterns.

4.4.2 Training Objective. For assignment generation, we apply teacher forcing and average the negative log-likelihood over all valid target positions in a mini-batch. Let $\mathcal{I}$ denote the dispatch batches in the mini-batch and $Z_{\text{gen}} = \sum_{t \in \mathcal{I}} K_t^*$ the number of valid target positions. The generation loss is

$$\mathcal{L}_{\text{gen}} = -\frac{1}{Z_{\text{gen}}} \sum_{t \in \mathcal{I}} \sum_{k=1}^{K_t^*} \log \Pr(e_{t,*}^{(k)} \mid \mathcal{G}_t^{(k)}, e_{t,*}^{(<k)}). \quad (17)$$

For service-process learning, let $\mathcal{T} = \{\text{DA}, \text{PCAA}, \text{DCAA}\}$ be the auxiliary-task set, and let $\text{BCE}(\cdot, \cdot)$ denote binary cross-entropy.

For task $m \in \mathcal{T}$, y_{tr}^m is the event label of candidate e_r in batch t , and $\hat{p}_{tr}^m$ is its predicted event probability. We define $w_{tr} = 1$ if e_r was broadcast and its service-process feedback is observable, and $w_{tr} = 0$ otherwise. Let $Z_{\text{obs}} = \sum_{t \in \mathcal{I}} \sum_{r=1}^{P_t} w_{tr}$ be the number of candidates with observed feedback. The auxiliary loss is

$$\mathcal{L}_{\text{mtl}} = \frac{1}{Z_{\text{obs}}} \sum_{t \in \mathcal{I}} \sum_{r=1}^{P_t} w_{tr} \sum_{m \in \mathcal{T}} \lambda_m \text{BCE}(\hat{p}_{tr}^m, y_{tr}^m), \quad (18)$$

where λ_m controls the relative weight of task m . Let λ_{mtl} balance service-process learning against assignment generation. The final objective is

$$\mathcal{L} = \mathcal{L}_{\text{gen}} + \lambda_{\text{mtl}} \mathcal{L}_{\text{mtl}}. \quad (19)$$

Both losses are normalized by valid supervision units. We set $\lambda_{\text{mtl}} = 10$ based on the sensitivity analysis in Appendix A.5. Appendix B.2 provides the complete training and inference procedures.

5 Experiments

5.1 Experimental Settings

Datasets. The experiments cover five cities across DiDi’s international ride-hailing markets. Offline evaluation uses City I–III, while online A/B tests use City III–V; City III appears in both settings. The upstream retrieval system is fixed, so all methods receive candidate sets constructed under the same feasibility rules. Detailed city statistics are provided in Appendix A.1.

Offline evaluation uses simulation environments constructed from historical dispatch logs, with 14 days for training and 7 days for evaluation. Each simulator replays real order requests, driver states, and candidate connections and advances according to the generated decisions.

Online A/B tests run under live production traffic for 14 days using a 1-hour time-slice interleaved design. The Production Dispatching Pipeline (PDP) is the deployed multi-stage baseline; its Kuhn–Munkres variant, denoted PDP_{KM} , serves as the control. Results are reported as treatment–control (T–C) deltas.

Metrics. We use Answer Ratio (AR), Completion Ratio (CR), Average Pickup Time (APT), and Gross Merchandise Volume (GMV). Higher AR, CR, and GMV and lower APT indicate better performance. Detailed metric definitions are provided in Appendix A.1.

Baselines. We compare GenMatch with PDP_{KM} and two solver variants, $\text{PDP}_{\text{Greedy}}$ and PDP_{GS} , where GS denotes Gale–Shapley matching [4]; end-to-end MICOD method D2SN [34]; value-based methods V1D3 [22] and RLW [18]; and multi-agent methods CoRide [9] and CoopRide [25]. We also evaluate $\text{GenMatch}_{\text{Value}}$ to isolate pair-level prediction. Detailed definitions are provided in Appendix A.1.

Implementation. All offline results are averaged over five independent runs. Hardware, optimization, and model configurations are provided in Appendix B.1.

5.2 Offline Performance

Table 1 shows that GenMatch consistently outperforms the production and research baselines. Relative to PDP_{KM} , it improves AR, CR, and GMV by 0.31%–0.83%, 0.23%–1.17%, and 0.11%–0.55%, respectively, while reducing APT by 0.23%–0.72%. D2SN also generates assignments sequentially but remains inferior on every metric, indicating that generation alone is insufficient without explicit batch

Table 1: Offline performance relative to PDP_{KM}. Values are the mean $\pm$ standard deviation of percentage changes over five runs; bold and underline denote the best and second-best results.

Variant	City I				City II				City III			
	AR (%) $\uparrow$	CR (%) $\uparrow$	APT (%) $\downarrow$	GMV (%) $\uparrow$	AR (%) $\uparrow$	CR (%) $\uparrow$	APT (%) $\downarrow$	GMV (%) $\uparrow$	AR (%) $\uparrow$	CR (%) $\uparrow$	APT (%) $\downarrow$	GMV (%) $\uparrow$
PDP _{KM}	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	<u>0.00 $\pm$ 0.00</u>	0.00 $\pm$ 0.00	<u>0.00 $\pm$ 0.00</u>	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
PDP _{Greedy}	-1.86 $\pm$ 0.05	-1.69 $\pm$ 0.09	-0.24 $\pm$ 0.02	-0.14 $\pm$ 0.01	-0.97 $\pm$ 0.06	-1.26 $\pm$ 0.04	<u>-0.25 $\pm$ 0.01</u>	-1.16 $\pm$ 0.06	-1.92 $\pm$ 0.15	-2.35 $\pm$ 0.18	<u>-0.31 $\pm$ 0.02</u>	-0.19 $\pm$ 0.02
PDP _{GS}	-0.42 $\pm$ 0.02	-0.30 $\pm$ 0.03	-0.06 $\pm$ 0.01	-0.04 $\pm$ 0.00	-0.38 $\pm$ 0.02	-0.54 $\pm$ 0.02	+0.10 $\pm$ 0.01	-0.47 $\pm$ 0.04	-0.79 $\pm$ 0.06	-0.76 $\pm$ 0.07	+0.14 $\pm$ 0.01	-0.27 $\pm$ 0.02
D2SN	-0.09 $\pm$ 0.03	+0.18 $\pm$ 0.07	-0.36 $\pm$ 0.02	-0.05 $\pm$ 0.00	-0.03 $\pm$ 0.09	-0.19 $\pm$ 0.11	-0.12 $\pm$ 0.01	-0.47 $\pm$ 0.06	-0.37 $\pm$ 0.12	-0.44 $\pm$ 0.07	-0.03 $\pm$ 0.01	-0.24 $\pm$ 0.04
RLW	-0.78 $\pm$ 0.04	-0.50 $\pm$ 0.05	+0.42 $\pm$ 0.02	-0.02 $\pm$ 0.00	-0.24 $\pm$ 0.03	-0.48 $\pm$ 0.02	+0.34 $\pm$ 0.03	-0.27 $\pm$ 0.03	-0.68 $\pm$ 0.08	-0.71 $\pm$ 0.08	+0.22 $\pm$ 0.03	-0.16 $\pm$ 0.02
V1D3	-1.07 $\pm$ 0.03	-0.81 $\pm$ 0.07	+0.29 $\pm$ 0.01	-0.06 $\pm$ 0.01	-0.39 $\pm$ 0.04	-0.62 $\pm$ 0.03	+0.26 $\pm$ 0.02	-0.57 $\pm$ 0.05	-0.93 $\pm$ 0.10	-1.06 $\pm$ 0.11	+0.18 $\pm$ 0.02	-0.18 $\pm$ 0.02
CoRide	-1.51 $\pm$ 0.09	-1.27 $\pm$ 0.14	+0.98 $\pm$ 0.03	-0.13 $\pm$ 0.02	-0.72 $\pm$ 0.08	-1.02 $\pm$ 0.11	+0.67 $\pm$ 0.03	-0.96 $\pm$ 0.06	-2.17 $\pm$ 0.21	-2.54 $\pm$ 0.23	+0.47 $\pm$ 0.04	-0.43 $\pm$ 0.04
CoopRide	-1.13 $\pm$ 0.07	-0.93 $\pm$ 0.11	+0.77 $\pm$ 0.03	-0.10 $\pm$ 0.01	-0.55 $\pm$ 0.06	-0.74 $\pm$ 0.09	+0.58 $\pm$ 0.02	-0.76 $\pm$ 0.05	-1.77 $\pm$ 0.14	-1.33 $\pm$ 0.13	+0.43 $\pm$ 0.04	-0.33 $\pm$ 0.03
GenMatch _{Value}	+0.29 $\pm$ 0.02	<u>+0.20 $\pm$ 0.03</u>	-0.76 $\pm$ 0.02	-0.02 $\pm$ 0.00	+0.14 $\pm$ 0.02	-0.09 $\pm$ 0.01	-0.19 $\pm$ 0.01	+0.03 $\pm$ 0.01	<u>+0.46 $\pm$ 0.05</u>	<u>+0.67 $\pm$ 0.06</u>	-0.58 $\pm$ 0.05	+0.24 $\pm$ 0.02
GenMatch	+0.51 $\pm$ 0.03	+0.62 $\pm$ 0.04	<u>-0.72 $\pm$ 0.03</u>	+0.11 $\pm$ 0.01	+0.31 $\pm$ 0.03	+0.23 $\pm$ 0.02	-0.40 $\pm$ 0.03	+0.23 $\pm$ 0.03	+0.83 $\pm$ 0.07	+1.17 $\pm$ 0.12	-0.23 $\pm$ 0.03	+0.55 $\pm$ 0.03

Table 2: Core City III ablations relative to GenMatch (Full), reported as the mean $\pm$ standard deviation of percentage changes over five runs. Bold denotes the best result in each column.

Module	Variant	AR $\uparrow$	CR $\uparrow$	APT $\downarrow$	GMV $\uparrow$
GenMatch	Full	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
Encoder	A1	-2.55 $\pm$ 0.12	-3.07 $\pm$ 0.07	+1.16 $\pm$ 0.02	-2.28 $\pm$ 0.15
	A2	-2.43 $\pm$ 0.06	-2.69 $\pm$ 0.03	+1.14 $\pm$ 0.01	-2.15 $\pm$ 0.06
	A3	-0.39 $\pm$ 0.06	-0.25 $\pm$ 0.03	+0.18 $\pm$ 0.02	-0.40 $\pm$ 0.07
Learner	A4	-2.76 $\pm$ 0.16	-3.15 $\pm$ 0.08	+1.17 $\pm$ 0.03	-2.32 $\pm$ 0.18
	A5	-2.02 $\pm$ 0.08	-2.34 $\pm$ 0.04	+1.53 $\pm$ 0.02	-1.88 $\pm$ 0.09
	A8	-1.11 $\pm$ 0.04	-1.07 $\pm$ 0.02	+0.86 $\pm$ 0.01	-1.45 $\pm$ 0.05
Decoder	A9	-1.22 $\pm$ 0.14	-0.96 $\pm$ 0.09	+0.48 $\pm$ 0.04	-0.78 $\pm$ 0.19
	A12	-2.46 $\pm$ 0.15	-3.07 $\pm$ 0.20	+0.16 $\pm$ 0.03	-0.91 $\pm$ 0.05

interactions, business guidance, and supervised outcome learning. The value-based and multi-agent baselines likewise cannot consistently improve the current batch assignment.

GenMatch_{Value}, which replaces only PDP’s prediction stage, improves most metrics and validates the learned service-process predictions. Full GenMatch further improves AR, CR, and GMV by replacing value calculation and matching with state-aware generation, directly supporting our cross-stage inconsistency motivation. Appendix A.3 further evaluates the auxiliary predictions.

5.3 Ablation Study

Table 2 reports core ablations on City III, organized by module to test whether each targeted design contributes to the final assignment. For the encoder, A1 removes batch-level matching and competition information, while A2 restores matching information alone. A2 improves AR, CR, and GMV over A1, and full GenMatch further improves all metrics, validating both forms of context. Removing the shared degree embedding (A3) also degrades every metric, showing that neighborhood size complements attention-based neighbor composition.

For the learner, removing auxiliary supervision (A4) causes the largest AR and CR drops and reduces GMV by 2.32%, while removing the utility logit (A5) produces the largest APT increase; both representation supervision and business guidance are therefore necessary. For the decoder, removing residual-state or progress information (A8–A9) consistently hurts performance. A12 uses the

first-step generation logits as fixed Kuhn–Munkres weights and reduces AR and CR by 2.46% and 3.07%, showing that static scores cannot replace state-aware sequential generation. Appendix A.2 reports the remaining variants and complete three-city results.

5.4 Online A/B Testing

Overall online performance. Table 3 shows significant improvements in every city. Overall, GenMatch increases AR, CR, and GMV by 2.26%, 3.86%, and 2.97%, respectively, while reducing APT by 1.84%. GenMatch_{Value} also improves all four metrics over PDP_{KM}, showing that the service-process predictions learned by GenMatch provide more effective pair-level inputs to the production pipeline. Full GenMatch further improves AR, CR, and GMV over GenMatch_{Value} by 1.49%, 1.93%, and 1.48% overall. The offline and online results therefore show the same pattern: better pair-level predictions help, while end-to-end generation yields the strongest overall gains. This consistency supports our cross-stage objective inconsistency motivation and shows that the offline optimization transfers to live traffic.

Dispatch effectiveness and user experience. Table 4 provides a finer-grained view of the dispatch funnel and service quality. Broadcast Count decreases by 0.17%, while Answer Count and Completion Count increase by 2.16% and 3.84%, respectively. This result is consistent with the greedy autoregressive policy of GenMatch: it suppresses broadcasts that are unlikely to yield an answer or completion and generates more effective assignments. All passenger and driver experience metrics also improve. PBE decreases by 15.17%, PCBA and PCAA decrease by 9.28% and 7.61%, Driver Income and DA increase by 2.99% and 13.96%, and DCAA decreases by 6.99%. Passenger and driver willingness is highly uncertain in international markets. The conventional paradigm predicts these signals separately and then combines them through a hand-crafted value function, so errors can propagate across stages. GenMatch instead learns from heterogeneous behavioral feedback and uses it to guide the final assignment directly, improving both dispatch effectiveness and user experience.

Performance across supply–demand periods. Figure 3 shows that GenMatch remains effective under different supply–demand conditions and delivers larger gains in busier periods. From low-demand to peak-demand periods, the CR improvement rises from 3.24% to 4.12%. The reductions in PCAA and DCAA also expand from 1.76% and 3.48% to 8.26% and 7.96%, respectively. This trend is

Table 3: Online A/B test improvements over PDP_{KM} (T-C). Overall averages the three cities; * indicates $p < 0.05$.

Variant	City III				City IV				City V				Overall			
	AR $\uparrow$	CR $\uparrow$	APT $\downarrow$	GMV $\uparrow$	AR $\uparrow$	CR $\uparrow$	APT $\downarrow$	GMV $\uparrow$	AR $\uparrow$	CR $\uparrow$	APT $\downarrow$	GMV $\uparrow$	AR $\uparrow$	CR $\uparrow$	APT $\downarrow$	GMV $\uparrow$
GenMatch _{Value}	0.88%	2.31%*	-7.08%*	2.35%*	0.59%	3.06%*	-4.55%*	2.43%*	0.75%*	1.67%*	-4.05%*	1.04%	0.77%*	1.93%*	-4.85%*	1.49%*
GenMatch	3.18%*	5.37%*	-2.20%*	4.89%*	1.72%*	4.51%*	-1.68%*	3.93%*	2.01%*	3.26%*	-1.76%*	2.16%*	2.26%*	3.86%*	-1.84%*	2.97%*

Table 4: Online changes in dispatch effectiveness and experience relative to PDP_{KM} (T-C); * indicates $p < 0.05$.

Metric	Delta (T-C)
Dispatch Effectiveness Measures	
Broadcast Count	-0.17%*
Answer Count ($\uparrow$)	2.16%*
Completion Count ($\uparrow$)	3.84%*
Passenger Experience Measures	
Passenger Bad Experience Ratio (PBE) ($\downarrow$)	-15.17%*
Passenger Cancel Before Answer Ratio (PCBA) ($\downarrow$)	-9.28%*
Passenger Cancel After Answer Ratio (PCAA) ($\downarrow$)	-7.61%*
Driver Experience Measures	
Driver Income ($\uparrow$)	2.99%*
Driver Answer Ratio (DA) ($\uparrow$)	13.96%*
Driver Cancel After Answer Ratio (DCAA) ($\downarrow$)	-6.99%*

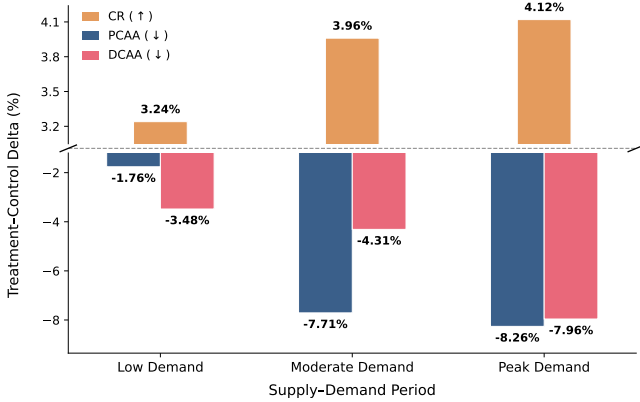


Figure 3: Online gains of GenMatch across supply-demand periods (T-C vs. PDP_{KM}).

particularly notable because the generation targets contain only OD pairs from rides that were actually completed online, as described in Section 4. Unbroadcast candidates have no observed service-process labels and are excluded from auxiliary supervision, but remain part of the complete dispatch batch and the generative candidate set. GenMatch can therefore transfer the completion patterns learned from completed pairs to previously unbroadcast candidates with similar structural and business characteristics. This ability becomes more valuable in peak periods, where denser competition leaves more latent completion opportunities unexplored, and converts them into additional completed rides and business gains.

6 Deployment

The existing Pair-Level Dispatch Engine processes OD pairs independently and therefore cannot provide the complete dispatch

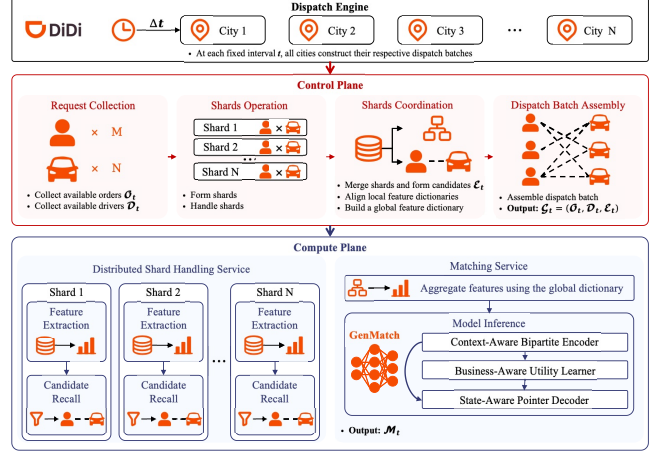


Figure 4: Production architecture of the Batch-Level Generative Dispatch Engine.

batch required by GenMatch. We develop a Batch-Level Generative Dispatch Engine for production serving. As shown in Figure 4, its control plane coordinates distributed shards, restores a consistent candidate order, and assembles the complete batch. Its compute plane retains distributed feature extraction and candidate retrieval, then performs global GenMatch inference over the assembled batch. Generated pairs are treated as pre-assignments and enter the existing arbitration and locking process, preventing conflicts with other product lines. Malformed outputs, timeouts, or serving failures automatically fall back to the pair-level engine, allowing GenMatch to be deployed without weakening production reliability. Appendix B.3 provides the engineering details.

7 Conclusion

This paper presents GenMatch, an end-to-end generative framework deployed in a real-world production environment for MICOD. By directly generating the final assignment, GenMatch addresses the cross-stage objective inconsistency of the conventional multi-stage paradigm. Its Context-Aware Bipartite Encoder, Business-Aware Utility Learner, and State-Aware Pointer Decoder address the three challenges of encoding dynamic batch-level structures, learning unified business utility from heterogeneous feedback, and generating assignments under an evolving matching state, respectively. Extensive offline experiments validate these targeted designs, while online A/B tests demonstrate the effectiveness and practicality of GenMatch in production.

References

- [1] Ben Chen, Xian Guo, Siyuan Wang, Zihan Liang, Yue Lv, Yufei Ma, Xinlong Xiao, Bowen Xue, Xuxin Zhang, Ying Yang, et al. 2025. Onesearch: A preliminary exploration of the unified end-to-end generative framework for e-commerce search. *arXiv preprint arXiv:2509.03236* (2025).
- [2] Ben Chen, Siyuan Wang, Yufei Ma, Zihan Liang, Xuxin Zhang, Yue Lv, Ying Yang, Huangyu Dai, Lingtao Mao, Tong Zhao, et al. 2026. OneSearch-V2: The Latent Reasoning Enhanced Self-distillation Generative Search Framework. *arXiv preprint arXiv:2603.24422* (2026).
- [3] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. Onerec: Unifying retrieve and rank with generative recommender and iterative preference alignment. *arXiv preprint arXiv:2502.18965* (2025).
- [4] David Gale and Lloyd S Shapley. 1962. College admissions and the stability of marriage. *The American mathematical monthly* 69, 1 (1962), 9–15.
- [5] Xian Guo, Ben Chen, Siyuan Wang, Ying Yang, Mingyue Cheng, Chenyi Lei, Yuqing Ding, and Han Li. 2026. Onesug: The unified end-to-end generative framework for e-commerce query suggestion. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 14774–14782.
- [6] Ruidong Han, Bin Yin, Shangyu Chen, He Jiang, Fei Jiang, Xiang Li, Chi Ma, Mincong Huang, Xiaoguang Li, Chunzhen Jing, et al. 2025. Mtgr: Industrial-scale generative recommendation framework in meituan. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 5731–5738.
- [7] Xiao Han, Zijian Zhang, Xiangyu Zhao, Yuanshao Zhu, Guojiang Shen, Xiangjie Kong, Xuetao Wei, Liqiang Nie, and Jieping Ye. 2025. Garlic: Gpt-augmented reinforcement learning with intelligent control for vehicle dispatching. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 255–263.
- [8] Yupeng Hou, Jiacheng Li, Ashley Shin, Jinsung Jeon, Abhishek Santhanam, Wei Shao, Kaveh Hassani, Ning Yao, and Julian McAuley. 2025. Generating long semantic ids in parallel for recommendation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 956–966.
- [9] Jiarui Jin, Ming Zhou, Weinan Zhang, Minne Li, Zilong Guo, Zhiwei Qin, Yan Jiao, Xiaocheng Tang, Chenxi Wang, Jun Wang, et al. 2019. Coride: joint order dispatching and fleet management for multi-scale ride-hailing platforms. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1983–1992.
- [10] Harold W Kuhn. 1955. The Hungarian method for the assignment problem. *Naval research logistics quarterly* 2, 1-2 (1955), 83–97.
- [11] Changhao Li, Junwei Yin, Zhilin Zeng, Senjie Kou, Shuli Wang, Wenshuai Chen, Yinhua Zhu, Haitao Wang, and Xingxing Wang. 2026. MBGR: Multi-Business Prediction for Generative Recommendation at Meituan. *arXiv preprint arXiv:2604.02684* (2026).
- [12] Hanyu Li, Yi-Ping Hsu, Aditya Mantha, Prabhat Agarwal, Laksh Basin, Jialu Wang, Hongtao Lin, Bella Huang, Yaxin Li, Xinyi Li, et al. 2026. UniPinRec: Unifying Generative Retrieval and Ranking at Pinterest Scale. *arXiv preprint arXiv:2606.00422* (2026).
- [13] Yile Liang, Jiuxia Zhao, Donghui Li, Jie Feng, Chen Zhang, Xuetao Ding, Jinghua Hao, and Renqing He. 2024. Harvesting efficient on-demand order pooling from skilled couriers: Enhancing graph representation learning for refining real-time many-to-one assignments. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5363–5374.
- [14] James Munkres. 1957. Algorithms for the assignment and transportation problems. *Journal of the society for industrial and applied mathematics* 5, 1 (1957), 32–38.
- [15] Zhiwei Qin, Xiaocheng Tang, Yan Jiao, Fan Zhang, Zhe Xu, Hongtu Zhu, and Jieping Ye. 2020. Ride-hailing order dispatching at didi via reinforcement learning. *INFORMS Journal on Applied Analytics* 50, 5 (2020), 272–286.
- [16] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems* 36 (2023), 10299–10315.
- [17] Xuxin Ren, Qiya Yang, Yichun Wu, Wei Xu, Yalong Wang, and Zhiqiang Zhang. 2024. Non-autoregressive generative models for reranking recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5625–5634.
- [18] Soheil Sadeghi Eshkevari, Xiaocheng Tang, Zhiwei Qin, Jinhan Mei, Cheng Zhang, Qianying Meng, and Jia Xu. 2022. Reinforcement learning in the wild: Scalable RL dispatching algorithm deployed in ridehailing marketplace. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 3838–3848.
- [19] Jaidev Shah, Iman Barjasteh, Amey Barapatre, Rana Forsati, Gang Luo, Fan Wu, Yuan Fang, Xue Deng, Blake Shepard, Ronak Shah, et al. 2025. Towards Web-scale Recommendations with LLMs: From Quality-aware Ranking to Candidate Generation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 2514–2524.
- [20] Yu Sun, Kai Wang, Zhipeng Hu, Runze Wu, Yaoxin Wu, Wen Song, Xudong Shen, Tangjie Lv, and Changjie Fan. 2024. MGMatch: Fast Matchmaking with Nonlinear Objective and Constraints via Multimodal Deep Graph Learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5741–5751.
- [21] Xiaocheng Tang, Zhiwei Qin, Fan Zhang, Zhaodong Wang, Zhe Xu, Yintai Ma, Hongtu Zhu, and Jieping Ye. 2019. A deep value-network based approach for multi-driver order dispatching. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 1780–1790.
- [22] Xiaocheng Tang, Fan Zhang, Zhiwei Qin, Yansheng Wang, Dingyuan Shi, Bingchen Song, Yongxin Tong, Hongtu Zhu, and Jieping Ye. 2021. Value function is all you need: A unified learning framework for ride hailing platforms. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 3605–3615.
- [23] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. 2015. Pointer networks. *Advances in neural information processing systems* 28 (2015).
- [24] Dongsheng Wang, Yuxi Huang, Shen Gao, Yifan Wang, Chengrui Huang, and Shuo Shang. 2025. Generative next poi recommendation with semantic id. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 2904–2914.
- [25] Jingwei Wang, Qianyu Hao, Wenzhen Huang, Xiaochen Fan, Qin Zhang, Zhen-tao Tang, Bin Wang, Jianye Hao, and Yong Li. 2025. Coopride: Cooperate all grids in city-scale ride-hailing dispatching with multi-agent reinforcement learning. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 1457–1468.
- [26] Yuxiang Wang, Chi Ma, Xiao Yan, Mincong Huang, Xiaoguang Li, Ruidong Han, Bin Yin, Shangyu Chen, Xiang Li, Fei Jiang, et al. 2026. MTGenRec: An Efficient Distributed Training System for Generative Recommendation Models in Meituan. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 2482–2493.
- [27] Ye Wang, Jiahao Xun, Minjie Hong, Jieming Zhu, Tao Jin, Wang Lin, Haoyuan Li, Linjun Li, Yan Xia, Zhou Zhao, et al. 2024. Eager: Two-stream generative recommender with behavior-semantic collaboration. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3245–3254.
- [28] Zhipeng Wei, Kuo Cai, Junda She, Jie Chen, Minghao Chen, Yang Zeng, Qiang Luo, Wencong Zeng, Ruiming Tang, Kun Gai, et al. 2026. Oneloc: Geo-aware generative recommender systems for local life service. In *Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining*. 735–744.
- [29] Zhe Xu, Zhixin Li, Qingwen Guan, Dingshui Zhang, Qiang Li, Junxiao Nan, Chunyang Liu, Wei Bian, and Jieping Ye. 2018. Large-scale order dispatch in on-demand ride-hailing platforms: A learning and planning approach. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 905–913.
- [30] Ben Xue, Dan Liu, Lixiang Wang, Mingjie Sun, Peng Wang, Pengfei Zhang, Shaoyun Shi, Tianyu Xu, Yunhao Sha, Zhiqiang Liu, et al. 2026. Generative recommendation for large-scale advertising. *arXiv preprint arXiv:2602.22732* (2026).
- [31] Zhaoxing Yang, Haiming Jin, Guiyun Fan, Min Lu, Yiran Liu, Xinlang Yue, Hao Pan, Zhe Xu, Guobin Wu, Qun Li, et al. 2024. Rethinking order dispatching in online ride-hailing platforms. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*. 3863–3873.
- [32] Junhao Yin, Haolin Wang, Peng Bao, Ju Xu, and Yongliang Wang. 2026. From clicks to preference: A multi-stage alignment framework for generative query suggestion in conversational system. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 2539–2550.
- [33] Yuanqing Yu, Yifan Wang, Weizhi Ma, Zhiqiang Guo, and Min Zhang. 2026. APAO: Bridging the Training-Inference Gap in Generative Recommendation via Adaptive Prefix-Aware Optimization. *arXiv preprint arXiv:2603.02730* (2026).
- [34] Xinlang Yue, Yiran Liu, Fangzhou Shi, Sihong Luo, Chen Zhong, Min Lu, and Zhe Xu. 2024. An end-to-end reinforcement learning based approach for micro-view order-dispatching in ride-hailing. In *Proceedings of the 33rd ACM international conference on information and knowledge management*. 5054–5061.
- [35] Ziyang Zeng, Heming Jing, Jindong Chen, Xiangli Li, Hongyu Liu, Yixuan He, Zhengyu Li, Yige Sun, Zheyong Xie, Yuqing Yang, et al. 2026. Optimizing Generative Ranking Relevance via Reinforcement Learning in Xiaohongshu Search. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 2551–2561.
- [36] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, et al. 2024. Actions speak louder than words: trillion-parameter sequential transducers for generative recommendations. In *Proceedings of the 41st International Conference on Machine Learning*. 58484–58509.
- [37] Hongbo Zhang, Guang Wang, Xu Wang, Zhengyang Zhou, Chen Zhang, Zheng Dong, and Yang Wang. 2024. Nondbrem: Nondeterministic offline reinforcement learning for large-scale order dispatching. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 401–409.
- [38] Libin Zheng, Lei Chen, and Jieping Ye. 2018. Order dispatch in price-aware ridesharing. *Proceedings of the VLDB Endowment* 11, 8 (2018), 853–865.

- [39] Zhuoxun Zheng, Baifan Zhou, Ahmet Soylu, Jie Tang, and Evgeny Kharlamov. 2026. DiKGRc: Generative Recommender Model with Diffusion and Knowledge Graph-Based Reasoning. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 2042–2053.
- [40] Guorui Zhou, Hengrui Hu, Hongtao Cheng, Huanjie Wang, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Lu Ren, Liao Yu, et al. 2025. Onerec-v2 technical report. *arXiv preprint arXiv:2508.20900* (2025).
- [41] Zhuang Zhuang, Shanshan Feng, Hangwei Qian, Mingqi Yang, Heng Qi, Yanming Shen, and Baocai Yin. 2026. Think2Go: Generative Next POI Recommendation with LLM Reasoning. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 2112–2123.
- [42] Yanyan Zou, Junbo Qi, Lunsong Huang, Yu Li, Kewei Xu, Jiabao Gao, Binglei Zhao, Xuanhua Yang, Sulong Xu, and Shengjie Li. 2026. GenRec: A Preference-Oriented Generative Framework for Large-Scale Recommendation. *arXiv preprint arXiv:2604.14878* (2026).

A Additional Experimental Details

A.1 Experimental Details

A.1.1 Datasets and City Statistics. We evaluate GenMatch in five cities from DiDi’s international ride-hailing markets. City I and City II are used for offline evaluation, City IV and City V are used for online evaluation, and City III is included in both settings to connect the offline and online observations. As shown in Table 5, the cities cover substantially different operating scales: daily completed orders range from approximately 1.7K to 13.8K, daily online drivers range from 0.18K to 0.68K, and average trip distance ranges from 3.47 km to 4.71 km. This diversity allows us to evaluate the method under different market sizes and supply–demand conditions rather than on a single operating environment.

For offline experiments, we construct city-specific simulation environments from historical dispatch logs. Fourteen days of data are used for training and seven days for evaluation. At every re-played dispatch step, the simulator provides the contemporaneous orders, available drivers, and feasible candidate connections. The upstream retrieval and feasibility-filtering system is fixed for all methods, so every method receives candidate sets generated under the same pickup-distance, service-range, and business constraints. Consequently, the comparison focuses on how each method represents, values, and selects from a common candidate set rather than on differences in candidate retrieval.

Each dispatch batch is represented as a sparse bipartite graph containing order nodes, driver nodes, and feasible OD-pair edges. An order is described by 113 features, a driver by 140 features, and a candidate edge by 32 features, giving 285 input fields in total. These fields cover spatiotemporal context, historical behavior, supply–demand statistics, and pair-specific business signals such as pickup cost and transaction value. The graph size changes across dispatch steps because the numbers of available orders, drivers, and feasible connections are determined by the current market state.

A.1.2 Metrics. We evaluate dispatch performance using Answer Ratio (AR), Completion Ratio (CR), Average Pickup Time (APT), and Gross Merchandise Volume (GMV). Together, these metrics describe whether an assignment is accepted, whether it is successfully fulfilled, how efficiently the driver reaches the passenger, and how much transaction value it creates.

Answer Ratio (AR) is the percentage of dispatched orders answered by drivers. A larger AR indicates that the selected OD pairs better match driver willingness and that fewer broadcasts are spent

on assignments unlikely to receive a response. It therefore measures the immediate effectiveness of the dispatch decision at the first stage of the service process.

Completion Ratio (CR) is the percentage of assigned orders that ultimately complete their trips. Compared with AR, CR additionally reflects cancellations and other failures after an answer. A larger CR indicates that the generated assignments are more likely to survive the complete service process and become successfully fulfilled rides, making it a direct measure of dispatch reliability and service conversion.

Average Pickup Time (APT) measures the average elapsed time from assignment to driver pickup. A smaller APT means that drivers can reach passengers more quickly, reducing passenger waiting time and driver-side pickup cost. APT therefore captures the spatial and operational efficiency of the selected matching rather than only whether the trip is answered or completed.

Gross Merchandise Volume (GMV) is the total transaction value contributed by completed trips. A larger GMV indicates that the dispatch policy produces greater realized business value, jointly reflecting the number of completed rides and their transaction values.

No single metric fully characterizes dispatch quality. For example, aggressively prioritizing nearby candidates may reduce APT without maximizing completion or transaction value, while prioritizing high-value trips alone may increase pickup cost. We therefore assess the four metrics jointly: higher AR, CR, and GMV and lower APT indicate better overall performance.

A.1.3 Baselines. We compare GenMatch with production, end-to-end, value-based, and multi-agent dispatch methods. All methods operate on candidate sets produced by the same upstream retrieval and feasibility-filtering system.

Production Dispatching Pipeline (PDP). PDP follows the conventional industrial paradigm: it first predicts business outcomes independently for each OD pair, combines these signals with a hand-crafted value function, and then constructs a batch-level assignment with a separate matching solver. PDP_{KM} is the deployed configuration and uses Kuhn–Munkres matching to optimize the assignment globally under the calculated pair weights. PDP_{Greedy} and PDP_{GS} retain the same upstream predictions and pair values but replace the final solver with greedy selection and Gale–Shapley matching, respectively. Comparing these variants isolates the effect of the matching solver within the conventional multi-stage pipeline.

End-to-End MICOD Baseline. D2SN formulates MICOD as a two-layer Markov decision process and sequentially selects OD pairs or hold actions with an encoder–decoder reinforcement-learning policy. It is the most closely related end-to-end baseline because it also constructs an assignment sequentially. Unlike GenMatch, however, it does not explicitly model candidate interactions on the sparse bipartite graph or use direct multi-task supervision from stage-wise service outcomes.

Value-Based Baselines. V1D3 and RLW improve dispatch decisions by learning or refining value estimates for candidate assignments. They represent approaches that enhance pair-level weights while retaining a value-driven decision pipeline. Their comparison with GenMatch tests whether improved pair values alone are

Table 5: Scale statistics of the five experimental cities.

Split	City	Daily Completed Orders	Daily Online Drivers	Avg. Trip Distance (m)
Offline	City I	3.04e+03	0.18e+03	3.47e+03
	City II	1.12e+04	0.55e+03	4.41e+03
Offline & Online	City III	5.91e+03	0.43e+03	3.99e+03
Online	City IV	1.73e+03	0.31e+03	4.29e+03
	City V	1.38e+04	0.68e+03	4.71e+03

Table 6: Complete ablation results relative to GenMatch (Full). Values are the mean $\pm$ standard deviation of percentage changes over five runs.

Module	Variant	City I				City II				City III			
		AR (%) $\uparrow$	CR (%) $\uparrow$	APT (%) $\downarrow$	GMV (%) $\uparrow$	AR (%) $\uparrow$	CR (%) $\uparrow$	APT (%) $\downarrow$	GMV (%) $\uparrow$	AR (%) $\uparrow$	CR (%) $\uparrow$	APT (%) $\downarrow$	GMV (%) $\uparrow$
GenMatch	Full	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
Encoder	A1	-2.32 $\pm$ 0.04	-2.61 $\pm$ 0.05	+1.74 $\pm$ 0.02	-0.57 $\pm$ 0.04	-2.14 $\pm$ 0.07	-2.55 $\pm$ 0.04	+1.23 $\pm$ 0.04	-2.07 $\pm$ 0.10	-2.55 $\pm$ 0.12	-3.07 $\pm$ 0.07	+1.16 $\pm$ 0.02	-2.28 $\pm$ 0.15
	A2	-2.25 $\pm$ 0.02	-2.30 $\pm$ 0.02	+1.61 $\pm$ 0.01	-0.52 $\pm$ 0.02	-1.95 $\pm$ 0.03	-2.36 $\pm$ 0.02	+1.06 $\pm$ 0.02	-1.96 $\pm$ 0.05	-2.43 $\pm$ 0.06	-2.69 $\pm$ 0.03	+1.14 $\pm$ 0.01	-2.15 $\pm$ 0.06
	A3	-0.14 $\pm$ 0.02	-0.12 $\pm$ 0.02	+0.13 $\pm$ 0.01	-0.04 $\pm$ 0.01	-0.66 $\pm$ 0.05	-1.04 $\pm$ 0.04	+0.49 $\pm$ 0.03	-0.72 $\pm$ 0.10	-0.39 $\pm$ 0.06	-0.25 $\pm$ 0.03	+0.18 $\pm$ 0.02	-0.40 $\pm$ 0.07
Learner	A4	-2.47 $\pm$ 0.05	-2.69 $\pm$ 0.06	+2.03 $\pm$ 0.01	-0.59 $\pm$ 0.05	-2.20 $\pm$ 0.09	-2.97 $\pm$ 0.06	+1.28 $\pm$ 0.05	-2.30 $\pm$ 0.15	-2.76 $\pm$ 0.16	-3.15 $\pm$ 0.08	+1.17 $\pm$ 0.03	-2.32 $\pm$ 0.18
	A5	-1.83 $\pm$ 0.03	-1.99 $\pm$ 0.03	+2.70 $\pm$ 0.01	-0.45 $\pm$ 0.03	-1.61 $\pm$ 0.05	-1.90 $\pm$ 0.03	+1.44 $\pm$ 0.03	-1.70 $\pm$ 0.08	-2.02 $\pm$ 0.08	-2.34 $\pm$ 0.04	+1.53 $\pm$ 0.02	-1.88 $\pm$ 0.09
Decoder	A6	-1.12 $\pm$ 0.02	-0.49 $\pm$ 0.02	+0.76 $\pm$ 0.01	-0.15 $\pm$ 0.01	-0.98 $\pm$ 0.04	-0.90 $\pm$ 0.02	+0.53 $\pm$ 0.02	-0.52 $\pm$ 0.03	-1.47 $\pm$ 0.07	-1.69 $\pm$ 0.04	+0.63 $\pm$ 0.02	-0.65 $\pm$ 0.04
	A7	-0.22 $\pm$ 0.01	-0.18 $\pm$ 0.01	+0.20 $\pm$ 0.00	-0.06 $\pm$ 0.01	-0.11 $\pm$ 0.02	-0.40 $\pm$ 0.01	+0.16 $\pm$ 0.01	-0.24 $\pm$ 0.03	-0.37 $\pm$ 0.05	-0.28 $\pm$ 0.02	+0.17 $\pm$ 0.01	-0.43 $\pm$ 0.04
	A8	-0.86 $\pm$ 0.01	-1.55 $\pm$ 0.01	+1.39 $\pm$ 0.00	-0.40 $\pm$ 0.01	-0.67 $\pm$ 0.02	-1.31 $\pm$ 0.01	+0.90 $\pm$ 0.01	-1.37 $\pm$ 0.04	-1.11 $\pm$ 0.04	-1.07 $\pm$ 0.02	+0.86 $\pm$ 0.01	-1.45 $\pm$ 0.05
	A9	-0.92 $\pm$ 0.03	-0.63 $\pm$ 0.04	+0.58 $\pm$ 0.02	-0.27 $\pm$ 0.04	-0.52 $\pm$ 0.07	-1.05 $\pm$ 0.05	+0.46 $\pm$ 0.04	-0.69 $\pm$ 0.14	-1.22 $\pm$ 0.14	-0.96 $\pm$ 0.09	+0.48 $\pm$ 0.04	-0.78 $\pm$ 0.19
	A10	-1.57 $\pm$ 0.06	-1.31 $\pm$ 0.07	+1.09 $\pm$ 0.02	-0.23 $\pm$ 0.07	-1.35 $\pm$ 0.11	-1.88 $\pm$ 0.07	+0.71 $\pm$ 0.06	-0.82 $\pm$ 0.20	-1.84 $\pm$ 0.20	-2.05 $\pm$ 0.10	+0.73 $\pm$ 0.04	-0.91 $\pm$ 0.24
	A11	-0.46 $\pm$ 0.07	-0.85 $\pm$ 0.08	+0.50 $\pm$ 0.03	-0.31 $\pm$ 0.09	-0.29 $\pm$ 0.13	-1.38 $\pm$ 0.08	+0.32 $\pm$ 0.07	-1.12 $\pm$ 0.25	-0.79 $\pm$ 0.24	-0.84 $\pm$ 0.12	+0.42 $\pm$ 0.05	-1.04 $\pm$ 0.30
	A12	-1.93 $\pm$ 0.15	-1.79 $\pm$ 0.10	+0.59 $\pm$ 0.03	-0.23 $\pm$ 0.01	-1.08 $\pm$ 0.11	-1.17 $\pm$ 0.14	+0.25 $\pm$ 0.03	-1.09 $\pm$ 0.07	-2.46 $\pm$ 0.15	-3.07 $\pm$ 0.20	+0.16 $\pm$ 0.03	-0.91 $\pm$ 0.05

Table 7: Auxiliary prediction AUC relative to PDP, reported as the mean $\pm$ standard deviation of percentage changes over five runs.

Variant	City I			City II			City III		
	AUC _{DA} (%) $\uparrow$	AUC _{PCAA} (%) $\uparrow$	AUC _{DCAA} (%) $\uparrow$	AUC _{DA} (%) $\uparrow$	AUC _{PCAA} (%) $\uparrow$	AUC _{DCAA} (%) $\uparrow$	AUC _{DA} (%) $\uparrow$	AUC _{PCAA} (%) $\uparrow$	AUC _{DCAA} (%) $\uparrow$
PDP	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
GenMatch _{MTL}	+1.62 $\pm$ 0.11	+0.84 $\pm$ 0.04	+0.74 $\pm$ 0.07	+1.79 $\pm$ 0.14	+0.76 $\pm$ 0.05	+0.45 $\pm$ 0.03	+1.74 $\pm$ 0.15	+0.82 $\pm$ 0.05	+0.55 $\pm$ 0.03

sufficient, or whether jointly learning batch context and the final assignment provides additional benefit.

Multi-Agent Baselines. CoRide and CoopRide model driver or regional cooperation with multi-agent reinforcement learning and are primarily designed to improve fleet-level, long-term supply-demand coordination. We include them to examine whether policies developed from a Macro-View perspective transfer effectively to the current-batch, OD-pair-level MICOD objective.

GenMatch_{Value}. This variant replaces only PDP’s pair-level prediction stage with the DA, PCAA, and DCAA predictions learned by GenMatch. The original hand-crafted value calculation and Kuhn-Munkres matching stages remain unchanged. It isolates the benefit of the Business-Aware Utility Learner’s service-process predictions, while the comparison between GenMatch_{Value} and full GenMatch measures the additional contribution of learned utility and state-aware assignment generation.

A.1.4 Evaluation Protocols. Offline evaluation replays seven days of historical dispatch traffic in each city-specific simulator after training on fourteen days of data. The replay preserves the observed order arrivals, driver states, and candidate connections, and all competing methods are evaluated under the same upstream feasibility rules. We report AR, CR, APT, and GMV relative to PDP_{KM}. Each offline result is averaged over five independent runs, and the

corresponding standard deviation reflects variation across those runs.

Online A/B tests are conducted in City III–V for fourteen days. GenMatch and the PDP_{KM} control are evaluated with a one-hour time-slice interleaved design, and results are reported as treatment-minus-control changes. This design compares the two systems under recurring live traffic conditions while limiting long-term drift between the treatment and control periods. The online evaluation uses the same four primary dispatch metrics as the offline study and additionally examines broadcast volume, answer and completion counts, passenger cancellation and bad-experience ratios, driver answer and cancellation ratios, and driver income.

A.1.5 Implementation Details. All models are trained on four NVIDIA RTX L20 GPUs. We use DeepSpeed Zero Redundancy Optimizer (ZeRO) Stage 2 to partition optimizer states and gradients across devices and bfloat16 (BF16) mixed precision to reduce memory and computation costs. The default GenMatch configuration contains two sparse bipartite encoder layers and two pointer-decoder layers with hidden dimension 128. Matching attention, competition attention, and pointer scoring each use four heads, and the feed-forward dimension is 512. The model supports at most 500 orders, 500 drivers, and 10,000 candidate OD pairs in one dispatch batch.

Training uses Adam for 50 epochs, a cosine learning-rate schedule with three warm-up epochs, a learning-rate search range of 5×10^{-5} to 5×10^{-4} , weight decay of 10^{-4} , and gradient clipping at 1.0. The per-GPU batch size is 16 and the global batch size is 64. The three auxiliary service-process tasks receive equal task weights, and the overall multi-task-loss coefficient is set to $\lambda_{\text{mtl}} = 10$ according to the sensitivity analysis in Section A.5. Offline experiments use five independent random seeds. Complete model and optimization settings are summarized in Table 9, and the training and inference procedures are provided in Section B.2.

A.2 Full Ablation Results

Table 6 reports all twelve ablations on the three offline cities. We analyze each variant together with its corresponding result and implication, organized around the three modules of GenMatch.

Context-Aware Bipartite Encoder (A1–A3). A1 removes both batch-level matching attention in Eq. (7) and competition attention in Eq. (6), reducing the encoder to isolated OD-pair representations. This change degrades every metric, decreasing AR by 2.14%–2.55%, CR by 2.55%–3.07%, and GMV by 0.57%–2.28%, while increasing APT by 1.16%–1.74%. A2 restores matching attention over the complete candidate neighborhoods while still removing competition attention. Relative to A1, it improves AR by 0.07%–0.19%, CR by 0.19%–0.38%, and GMV by 0.05%–0.13%, while reducing the APT increase by 0.02%–0.17%; hence, neighborhood-level matching information is useful even without competition modeling. Full GenMatch further improves all metrics over A2, confirming that competition relations provide complementary batch context. A3 instead retains both attention mechanisms but removes their shared degree embedding. Its effect is small in City I (−0.14% AR and −0.12% CR), becomes largest in City II (−0.66% AR, −1.04% CR, and −0.72% GMV), and is intermediate in City III. This city-dependent degradation indicates that explicit neighborhood size is particularly useful in larger candidate graphs, where normalized attention alone cannot preserve the scale of local competition.

Business-Aware Utility Learner (A4–A5). A4 removes the multi-task supervision of candidate memory while retaining the stage-wise predictions used to construct utility. It decreases AR by 2.20%–2.76%, CR by 2.69%–3.15%, and GMV by 0.59%–2.32%, producing the largest AR and CR degradations among the learner variants in every city. Thus, behavioral-outcome supervision improves not only the auxiliary predictions but also the shared candidate representations used for assignment. A5 retains this supervision but removes the utility logit from Eq. (14), leaving the decoder to rely on its structural score. It degrades all four metrics and increases APT by 1.44%–2.70%, the largest APT increase among all ablations in every city. This result shows that the structural score alone cannot recover the service-efficiency trade-offs captured by explicit business-value guidance.

State-Aware Pointer Decoder (A6–A12). A6, A7, and A8 remove the initial-, selected-, and residual-state representations, respectively, from the state-aware query in Eq. (12). A7 causes the smallest degradation of the three, whereas A6 reduces AR by 0.98%–1.47% and A8 reduces CR by 1.07%–1.55% and GMV by 0.40%–1.45%. The contrast shows that the original batch and, especially, the remaining feasible opportunities provide more decision context than the

already selected pairs, although all three summaries contribute. A9 removes progress encoding and consistently degrades every metric, including AR by 0.52%–1.22% and CR by 0.63%–1.05%. This confirms that the averages of the three state sets do not by themselves retain how far generation has progressed. A10 replaces the explicit state-aware query with causal self-attention; its AR and CR losses reach 1.84% and 2.05% in City III, showing that implicit history propagation does not represent the evolving feasible set as effectively. A11 trains with a fixed target order and remains inferior to Full in every city, with CR decreasing by 0.84%–1.38% and GMV by 0.31%–1.12%; random target permutations therefore reduce dependence on an arbitrary generation order. Finally, A12 uses the first-step generation logits as fixed weights for Kuhn–Munkres matching. It produces particularly large AR and CR losses, reaching −2.46% and −3.07%, respectively, in City III. Although its APT change is comparatively small, the loss in answered and completed orders demonstrates that frozen scores and one-shot optimization cannot replace the score updates required after each selected pair changes the feasible set.

Overall, every targeted removal degrades all four dispatch metrics across all three cities. The encoder results establish the complementary roles of neighborhood composition, competition context, and neighborhood size; the learner results validate behavioral supervision and explicit business guidance; and the decoder results show that assignment quality depends on dynamically representing and updating the matching state. These complete results support the conclusions drawn from the compact City III table in the main text.

A.3 Auxiliary Prediction Evaluation

We use the area under the receiver operating characteristic curve (AUC) to evaluate the auxiliary DA, PCAA, and DCAA predictions. A larger AUC indicates better discrimination between positive and negative outcomes. Table 7 reports relative AUC changes over the production prediction model.

GenMatch improves all three tasks in every city, demonstrating that the gains are consistent across different markets rather than being specific to one dataset. DA improves by 1.62%–1.79%, the largest gain among the three tasks. PCAA and DCAA improve by 0.76%–0.84% and 0.45%–0.74%, respectively. The relatively small standard deviations over five runs further indicate stable improvements.

These results verify that the Business-Aware Utility Learner captures the stage-wise service process more accurately than the production prediction model. They also explain the improvement of $\text{GenMatch}_{\text{Value}}$: replacing only the prediction stage with these auxiliary predictions already benefits dispatch performance, even when the hand-crafted value calculation and Kuhn–Munkres matching remain unchanged. The additional improvement of full GenMatch reported in the main text therefore comes from jointly learning business utility and generating the final assignment, rather than from prediction accuracy alone.

A.4 Effect of Model Capacity

Figure 5 studies model capacity by varying encoder depth, decoder depth, and hidden dimension d . Results are reported relative to the *Medium* configuration, and the outlined markers identify the

best configuration for each city. The *Small* model degrades all four metrics across all three cities, showing that insufficient capacity limits both matching quality and business value. Reducing encoder depth, decoder depth, or width also causes broad performance drops. A shallower encoder markedly reduces CR and increases APT, confirming the importance of sufficient capacity for batch-level graph encoding. A shallower decoder mainly hurts AR and CR, while reduced width weakens all four metrics.

Increasing capacity beyond *Medium* does not yield consistent gains. The *Large* model slightly improves AR in City II and City III and reduces APT in City I and City III. However, it substantially reduces CR and GMV in every city. In contrast, *Medium* achieves the best CR and GMV across all three cities and the best APT in City II. It therefore provides the best overall balance between model capacity and dispatch performance and is used as the default configuration.

A.5 Effect of the Multi-Task-Loss Weight

Table 8 studies λ_{mtl} , which balances the sequence-generation loss and the multi-task loss in Eq. (19). All values are reported relative to $\lambda_{\text{mtl}} = 10$. Weights below 10 provide insufficient behavioral supervision and consistently degrade dispatch performance. Larger weights can improve auxiliary AUC, especially at 1,000 and 10,000, but these prediction gains do not translate into better assignments: CR and GMV generally decrease because the multi-task objective begins to dominate sequence learning. We therefore set $\lambda_{\text{mtl}} = 10$, which provides the best overall balance across dispatch metrics and cities.

B Implementation and Deployment Details

B.1 Model Configuration

Table 9 lists the model and training configurations of GenMatch.

B.2 Training and Inference Procedures

Algorithm 1 summarizes training. Each dispatch batch is encoded once, after which the model learns stage-wise service outcomes and utility logits. A random permutation of the completed OD pairs provides the teacher-forced generation target, and the generation and auxiliary losses jointly update all model parameters.

Algorithm 2 details inference. GenMatch first computes pair memory and utility logits for the complete dispatch batch. At each step, it constructs the query from the current matching state, greedily selects the highest-probability selectable candidate, masks every conflicting candidate, and incrementally updates the selected and residual states. This loop continues while $\sum_r m_r^{(k)} > 0$, and the selected OD pairs form the final matching set.

B.3 Deployment Details

GenMatch has been deployed on a large-scale ride-hailing platform serving multiple international markets. The existing production system, which we call the Pair-Level Dispatch Engine, was designed for the conventional multi-stage paradigm. It partitions feasible order-driver pairs into shards, predicts pair-level business signals in parallel, calculates matching weights, and constructs the final assignment. This design is efficient because each pair can be processed independently before dispatch matching. It therefore avoids

Algorithm 1 GenMatch Training Procedure

Require: Mini-batches of $\{\mathcal{G}_t, C_t, \{y_r, w_r\}_{r=1}^{P_t}\}$; loss weights $\{\lambda_m\}_{m \in \mathcal{T}}$ and λ_{mtl}

Ensure: Trained model parameters Θ

- 1: Initialize model parameters Θ
- 2: **for** each training epoch **do**
- 3: **for** each mini-batch $\mathcal{I}$ **do**
- 4: Initialize accumulated losses and valid-position counts
- 5: **for** each dispatch batch $t \in \mathcal{I}$ **do**
- 6: Encode $\mathcal{G}_t$ as pair representations $\{z_r\}_{r=1}^{P_t}$ using Eq. (10)
- 7: Predict the DA, PCAA, and DCAA probabilities with the multi-task network
- 8: Obtain $\{a_r\}_{r=1}^{P_t}$ using Eq. (11)
- 9: Uniformly sample an ordering of C_t to form $\mathcal{Y}_t^* = (e_{t,*}^{(1)}, \dots, e_{t,*}^{(K_t^*)})$, where $K_t^* = |C_t|$
- 10: Set $S_{\text{init}} \leftarrow \sum_r z_r$, $N_{\text{init}} \leftarrow P_t$, and $\mathbf{q}_{\text{init}} \leftarrow S_{\text{init}}/N_{\text{init}}$
- 11: Initialize the selected-state statistics with $(S_{\text{sel}}^{(1)}, N_{\text{sel}}^{(1)}) \leftarrow (0, 0)$
- 12: Initialize the residual-state statistics with $(S_{\text{res}}^{(1)}, N_{\text{res}}^{(1)}) \leftarrow (S_{\text{init}}, P_t)$
- 13: Set $m_r^{(1)} \leftarrow 1$ for all r
- 14: **for** $k = 1, \dots, K_t^*$ **do**
- 15: Construct $\mathbf{q}^{(k)}$ using Eq. (12)
- 16: Compute $\{s_r^{(k)}\}$ and $\{p_r^{(k)}\}$ using Eqs. (13) and (14)
- 17: Under teacher forcing, find $r_*^{(k)}$ such that $e_{r_*^{(k)}} = e_{t,*}^{(k)}$
- 18: Accumulate $-\log p_{r_*^{(k)}}^{(k)}$
- 19: Let $\mathcal{B}^{(k)}$ contain the target pair and its currently selectable conflicting pairs
- 20: Update the selected- and residual-state statistics using Eq. (16), and mask $\mathcal{B}^{(k)}$
- 21: **end for**
- 22: **end for**
- 23: Compute $\mathcal{L}_{\text{gen}}$ and $\mathcal{L}_{\text{mtl}}$ using Eqs. (17) and (18)
- 24: Compute $\mathcal{L}$ using Eq. (19)
- 25: Update Θ using $\nabla_{\Theta} \mathcal{L}$
- 26: **end for**
- 27: **end for**
- 28: **return** Θ

assembling the complete dispatch batch during model inference and scales well under strict latency constraints.

GenMatch changes the serving unit from an individual pair to an entire dispatch batch. Its encoder requires the complete sparse bipartite graph, and its decoder must maintain a consistent candidate order while generating assignments. We therefore develop a Batch-Level Generative Dispatch Engine. This upgrade introduces three challenges. First, centralizing feature preparation would cause excessive compute and latency, while independent shards cannot provide the full batch structure. We retain distributed candidate filtering and feature preparation, then use a city-level orchestrator to restore candidate order and assemble the complete batch for

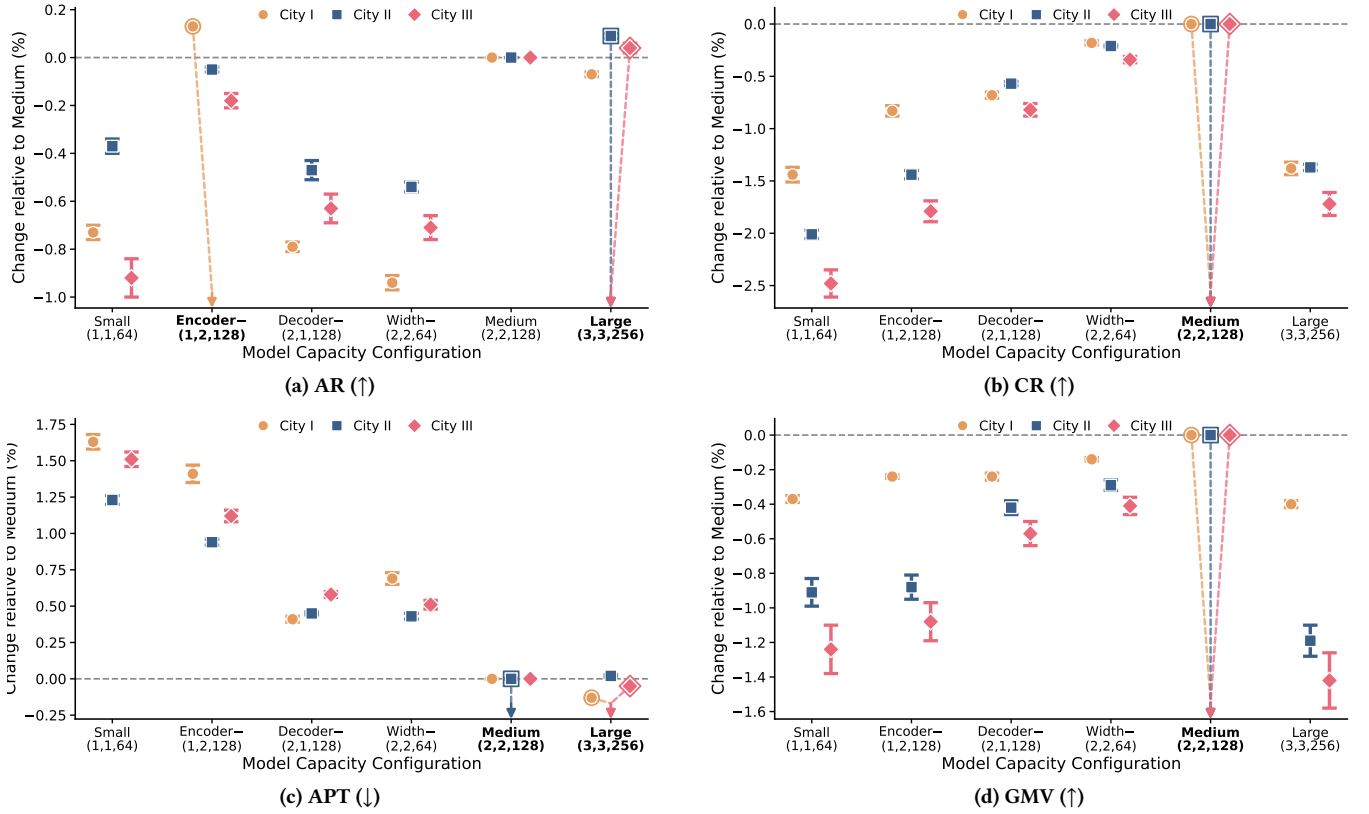


Figure 5: Effect of model capacity relative to the Medium configuration. Error bars denote standard deviations over five runs.

Table 8: Sensitivity to λ_{mtl} relative to the selected value 10. Values are the mean $\pm$ standard deviation of percentage changes over five runs. Larger AR, CR, GMV, and AUC and smaller APT are preferred. Bold and underlined values denote the best and second-best results in each column, respectively.

(a) Dispatch performance												
λ_{mtl}	City I				City II				City III			
	AR $\uparrow$	CR $\uparrow$	APT $\downarrow$	GMV $\uparrow$	AR $\uparrow$	CR $\uparrow$	APT $\downarrow$	GMV $\uparrow$	AR $\uparrow$	CR $\uparrow$	APT $\downarrow$	GMV $\uparrow$
0.01	-0.29 $\pm$ 0.05	-0.44 $\pm$ 0.06	+0.67 $\pm$ 0.01	-0.19 $\pm$ 0.05	-0.27 $\pm$ 0.05	-0.29 $\pm$ 0.06	+0.43 $\pm$ 0.01	-0.57 $\pm$ 0.05	-0.78 $\pm$ 0.05	-0.88 $\pm$ 0.06	+0.22 $\pm$ 0.01	-0.60 $\pm$ 0.05
0.1	-0.17 $\pm$ 0.02	-0.39 $\pm$ 0.02	+0.37 $\pm$ 0.01	-0.08 $\pm$ 0.02	-0.19 $\pm$ 0.02	-0.22 $\pm$ 0.02	+0.24 $\pm$ 0.01	-0.24 $\pm$ 0.02	-0.49 $\pm$ 0.02	-0.46 $\pm$ 0.02	+0.16 $\pm$ 0.01	-0.35 $\pm$ 0.02
1	-0.11 $\pm$ 0.03	-0.07 $\pm$ 0.03	+0.20 $\pm$ 0.01	-0.02 $\pm$ 0.03	-0.08 $\pm$ 0.03	-0.14 $\pm$ 0.03	+0.12 $\pm$ 0.01	-0.18 $\pm$ 0.03	-0.20 $\pm$ 0.03	-0.17 $\pm$ 0.03	+0.05 $\pm$ 0.01	-0.11 $\pm$ 0.03
10	<u>0.00$\pm$0.00</u>	<u>0.00$\pm$0.00</u>	0.00 $\pm$ 0.00	<u>0.00$\pm$0.00</u>	<u>0.00$\pm$0.00</u>	<u>0.00$\pm$0.00</u>	0.00 $\pm$ 0.00	<u>0.00$\pm$0.00</u>	<u>0.00$\pm$0.00</u>	<u>0.00$\pm$0.00</u>	0.00 $\pm$ 0.00	<u>0.00$\pm$0.00</u>
100	+0.21$\pm$0.05	-0.11 $\pm$ 0.06	+0.13 $\pm$ 0.01	+0.05$\pm$0.05	-0.07 $\pm$ 0.05	-0.16 $\pm$ 0.06	-0.13 $\pm$ 0.01	-0.06 $\pm$ 0.05	+0.08$\pm$0.05	-0.33 $\pm$ 0.06	-0.08 $\pm$ 0.01	-0.28 $\pm$ 0.05
1000	-0.30 $\pm$ 0.02	-0.19 $\pm$ 0.02	-0.56 $\pm$ 0.01	-0.05 $\pm$ 0.02	-0.21 $\pm$ 0.02	-0.19 $\pm$ 0.02	-0.36 $\pm$ 0.01	-0.30 $\pm$ 0.02	-0.15 $\pm$ 0.02	-0.39 $\pm$ 0.02	-0.16 $\pm$ 0.01	-0.35 $\pm$ 0.02
10000	-0.67 $\pm$ 0.03	-0.25 $\pm$ 0.03	-0.81$\pm$0.01	-0.17 $\pm$ 0.03	-0.42 $\pm$ 0.03	-0.32 $\pm$ 0.03	-0.47$\pm$0.01	-0.67 $\pm$ 0.03	-0.49 $\pm$ 0.03	-0.44 $\pm$ 0.03	-0.17$\pm$0.01	-0.41 $\pm$ 0.03
(b) Auxiliary prediction AUC												
λ_{mtl}	City I			City II			City III					
	DA	PCAA	DCAA	DA	PCAA	DCAA	DA	PCAA	DCAA			
0.01	-0.29 $\pm$ 0.05	+0.05 $\pm$ 0.06	+0.08 $\pm$ 0.01	-0.17 $\pm$ 0.05	-0.15 $\pm$ 0.06	-0.12 $\pm$ 0.01	-0.16 $\pm$ 0.05	-0.04 $\pm$ 0.06	-0.18 $\pm$ 0.01			
0.1	-0.17 $\pm$ 0.02	+0.11 $\pm$ 0.02	-0.14 $\pm$ 0.01	-0.14 $\pm$ 0.02	-0.04 $\pm$ 0.02	-0.05 $\pm$ 0.01	-0.14 $\pm$ 0.02	-0.16 $\pm$ 0.02	-0.11 $\pm$ 0.01			
1	-0.44 $\pm$ 0.03	+0.16 $\pm$ 0.03	-0.09 $\pm$ 0.01	-0.11 $\pm$ 0.03	-0.06 $\pm$ 0.03	-0.01 $\pm$ 0.01	-0.05 $\pm$ 0.03	-0.14 $\pm$ 0.03	-0.06 $\pm$ 0.01			
10	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00			
100	-0.06 $\pm$ 0.05	-0.06 $\pm$ 0.06	+0.03 $\pm$ 0.01	+0.09 $\pm$ 0.05	-0.03 $\pm$ 0.06	-0.04 $\pm$ 0.01	-0.07 $\pm$ 0.05	+0.09 $\pm$ 0.06	+0.13 $\pm$ 0.01			
1000	<u>+0.13$\pm$0.02</u>	<u>+0.18$\pm$0.02</u>	<u>+0.15$\pm$0.01</u>	+0.12$\pm$0.02	<u>+0.07$\pm$0.02</u>	<u>+0.03$\pm$0.01</u>	<u>+0.06$\pm$0.02</u>	+0.18$\pm$0.02	+0.19$\pm$0.01			
10000	+0.21$\pm$0.03	+0.24$\pm$0.03	+0.21$\pm$0.01	+0.08 $\pm$ 0.03	+0.14$\pm$0.03	+0.15$\pm$0.01	+0.12$\pm$0.03	<u>+0.15$\pm$0.03</u>	<u>+0.18$\pm$0.01</u>			

global inference. Second, GenMatch requires 285 features produced across different shards. Re-fetching them globally is costly, and

shard-local feature identifiers may be inconsistent. We pass sparse

Table 9: Model and training configurations of GenMatch.

Configuration	Symbol	Value
Encoder layers	L_{enc}	2
Decoder layers	L_{dec}	2
Hidden dimension	d	128
Matching-attention heads	–	4
Competition-attention heads	–	4
Pointer heads	P_{ptr}	4
Feed-forward dimension	–	512
Dropout ratio	–	0.2
Multi-task shared-layer dimensions	–	[256, 256]
Multi-task tower dimensions	–	[256, 128, 64]
DA loss weight	λ_{DA}	1.0
PCAA loss weight	λ_{PCAA}	1.0
DCAA loss weight	λ_{DCAA}	1.0
Multi-task-loss weight	λ_{mtl}	10.0
Maximum orders per batch	–	500
Maximum drivers per batch	–	500
Maximum candidate OD pairs	–	10000
Training epochs	–	50
Optimizer	–	Adam
Learning-rate range	–	$5 \times 10^{-5} - 5 \times 10^{-4}$
Learning-rate scheduler	–	Cosine
Warm-up epochs	–	3
Weight decay	–	10^{-4}
Batch size per GPU	–	16
Global batch size	–	64
Gradient clipping	–	1.0

Algorithm 2 GenMatch Inference Procedure**Require:** Dispatch batch $\mathcal{G}_t$; trained model parameters Θ **Ensure:** Generated assignment $\mathcal{M}_t$

- 1: Encode $\mathcal{G}_t$ as pair representations $\{\mathbf{z}_r\}_{r=1}^{P_t}$ using Eq. (10)
- 2: Predict the DA, PCAA, and DCAA probabilities with the multi-task network
- 3: Obtain $\{a_r\}_{r=1}^{P_t}$ using Eq. (11)
- 4: Initialize $\mathcal{M}_t \leftarrow \emptyset$, $m_r^{(1)} \leftarrow 1$ for all r , and $k \leftarrow 1$
- 5: Set $\mathbf{S}_{\text{init}} \leftarrow \sum_r \mathbf{z}_r$, $N_{\text{init}} \leftarrow P_t$, and $\mathbf{q}_{\text{init}} \leftarrow \mathbf{S}_{\text{init}}/N_{\text{init}}$
- 6: Initialize the selected-state statistics as $(\mathbf{0}, 0)$ and the residual-state statistics as $(\mathbf{S}_{\text{init}}, P_t)$
- 7: **while** $\sum_{r=1}^{P_t} m_r^{(k)} > 0$ **do**
- 8: Construct $\mathbf{q}^{(k)}$ using Eq. (12)
- 9: Compute $\{\bar{s}_r^{(k)}\}$ and $\{p_r^{(k)}\}$ using Eqs. (13) and (14)
- 10: Select $r^{(k)}$ using Eq. (15)
- 11: $\mathcal{M}_t \leftarrow \mathcal{M}_t \cup \{e_{r^{(k)}}\}$
- 12: Let $\mathcal{B}^{(k)}$ contain the selected pair and its currently selectable conflicting pairs
- 13: Update the selected- and residual-state statistics using Eq. (16), and mask $\mathcal{B}^{(k)}$
- 14: $k \leftarrow k + 1$
- 15: **end while**
- 16: **return** $\mathcal{M}_t$

features with their candidates and re-key them into one global feature dictionary during aggregation. Third, autoregressive outputs depend on stable candidate indices and must coexist with other product lines. We remove previously claimed candidates before inference and treat generated pairs as pre-assignments, which then enter the standard arbitration and locking process.

We deploy the new engine in three steps: feature transmission, shadow model inference, and generative decision making. Any serving failure, malformed output, or configuration error automatically falls back to the Pair-Level Dispatch Engine, and GenMatch can be disabled without redeployment. The resulting design retains the scalability of pair-level distributed processing while enabling safe batch-level generative matching in production.